
□ Topic: Arrays and Their Applications

Total MCQs: 30

Includes: CUET PG PYQs

Answer key: At the end

□ Basic Array Concepts

1. What is the time complexity to access an element in an array by index?
 - A. $O(n)$
 - B. $O(\log n)$
 - C. $O(1)$
 - D. $O(n^2)$☐
2. Which of the following is NOT a valid array operation?
 - A. Traversal
 - B. Insertion
 - C. Sorting
 - D. Deletion from middle in $O(1)$☐
3. Arrays store elements in:
 - A. Random memory locations
 - B. Linked nodes
 - C. Sequential memory locations
 - D. Stack-like structure☐
4. Which of the following sorting algorithms has the best average-case time complexity?
 - A. Bubble sort
 - B. Selection sort
 - C. Merge sort
 - D. Insertion sort☐
5. What will be the output of the array $A = [5, 2, 9, 1, 5]$ after bubble sort?
 - A. $[1, 2, 5, 5, 9]$
 - B. $[5, 2, 1, 5, 9]$

C. [9, 5, 5, 2, 1]

D. [5, 1, 2, 5, 9]

☐

☐ Array Applications

6. Arrays are best suited for:

- A. Frequent insertion/deletion
- B. Constant time element access
- C. Dynamic memory allocation
- D. Sparse data storage

☐

7. Which data structure is used to implement matrices efficiently?

- A. Stack
- B. Queue
- C. Array
- D. Tree

☐

8. Two-dimensional arrays are mainly used for:

- A. Queues
- B. Graphs
- C. Matrices
- D. Trees

☐

9. In a 2D array $A[3][4]$, how many total elements are stored?

- A. 7
- B. 12
- C. 3
- D. 4

☐

10. What is the index of the last element in a 1D array of size n ?

- A. n
- B. $n+1$
- C. $n-1$
- D. $n-2$

☐

☐ Searching in Arrays

11. What is the time complexity of linear search in the worst case?

- A. $O(\log n)$
- B. $O(1)$
- C. $O(n)$
- D. $O(n^2)$

☐

12. Binary search can only be applied to:

- A. Unsorted array
- B. Circular array
- C. Sorted array
- D. Linked list

☐

13. What is the average-case time complexity of binary search?

- A. $O(1)$
- B. $O(n)$
- C. $O(\log n)$
- D. $O(n \log n)$

☐

14. Which search algorithm is suitable for unsorted arrays?

- A. Binary search
- B. Linear search
- C. Interpolation search
- D. Exponential search

☐

15. What is the worst-case time complexity of binary search?

- A. $O(n)$
- B. $O(1)$
- C. $O(\log n)$
- D. $O(n \log n)$

☐

16. CUET PG PYQ: What is the output of the following C code?

```
int a[] = {1, 2, 3, 4};  
printf("%d", *(a+2));
```

- A. 1
- B. 2
- C. 3
- D. Error

☐

17. CUET PG PYQ: What is the result of accessing an index beyond the size of the array in C?

- A. Garbage value
- B. Compile-time error
- C. 0
- D. Program crash

☐

18. CUET PG PYQ: Which sorting algorithm uses divide and conquer strategy?

- A. Bubble sort
- B. Merge sort
- C. Selection sort
- D. Insertion sort

☐

19. CUET PG PYQ: What is the base index of arrays in C?

- A. 1
- B. 0
- C. -1
- D. Depends on compiler

☐

20. CUET PG PYQ: What is the correct way to declare a 2D array in C?

- A. int A(3,3);
- B. int A[3][3];
- C. int A{3,3};
- D. int A<3,3>;

☐

21. What does a[i] mean in arrays?

- A. Address of array
- B. Value at index i
- C. Pointer
- D. None of the above

☐

22. Arrays in C are:

- A. Dynamically allocated by default
- B. Statically allocated
- C. Not contiguous
- D. Allocated in stack always

☐

23. The size of an array must be known:

- A. At runtime
- B. At compile time
- C. Only in Java
- D. When creating pointer

☐

24. Which of these is used for array bounds checking in Java but not in C?

- A. Exception
- B. Garbage collector
- C. ArrayIndexOutOfBoundsException
- D. BufferOverflow

☐

25. In languages like C, if you access out-of-bounds index:

- A. Memory leak occurs
- B. It's automatically corrected
- C. Undefined behavior occurs
- D. Compiler throws error

☐

☐ Applications and Use-Cases

26. Which application commonly uses arrays?

- A. Expression parsing
- B. Polynomial representation

C. CPU job scheduling

D. All of the above

☐

27. Which of the following is NOT an application of arrays?

A. Hash tables

B. Queue

C. Stack

D. Tree traversal

☐

28. Sparse matrix is best stored using:

A. Array

B. Linked list

C. Hashing

D. Special representation

☐

29. Which of the following operations is inefficient on arrays?

A. Indexing

B. Appending at the end

C. Insertion in the middle

D. Traversal

☐

30. An array can be used to implement which of the following?

A. Stack

B. Queue

C. Matrix

D. All of the above

☐

☐ Answer Key

1-C 2-D 3-C 4-C 5-A

6-B 7-C 8-C 9-B 10-C

11-C 12-C 13-C 14-B 15-C

16-C 17-A 18-B 19-B 20-B

21-B 22-B 23-B 24-C 25-C

26-D 27-D 28-D 29-C 30-D

□ Topic: Sparse Matrix

Total MCQs: 30

Includes: Conceptual + Application-based + PYQs

Answer Key: Provided at the end

□ Basics of Sparse Matrix

1. What is a **sparse matrix**?
 - A. A matrix with mostly non-zero elements
 - B. A matrix with mostly zero elements
 - C. A matrix with only diagonal elements
 - D. A square matrix

□
2. If a matrix has **more than 90% zeros**, it is called:
 - A. Null matrix
 - B. Sparse matrix
 - C. Dense matrix
 - D. Zero matrix

□
3. Sparse matrices are used to:
 - A. Save space
 - B. Increase processing time
 - C. Make matrices square
 - D. Increase sparsity

□
4. Which of the following matrices is **not** a sparse matrix?
 - A. [0 0 1; 0 0 0; 0 0 0]
 - B. [1 1 1; 1 1 1; 1 1 1]
 - C. [0 0 0; 0 5 0; 0 0 0]
 - D. [0 0 0; 0 0 0; 0 0 9]

□

5. The space complexity of a dense $n \times n$ matrix is:
- A. $O(n)$
 - B. $O(\log n)$
 - C. $O(n^2)$
 - D. $O(1)$

☐

☐ Sparse Matrix Representations

6. Which of the following is a representation of a sparse matrix?
- A. Array of arrays
 - B. Triplet representation
 - C. Adjacency list
 - D. Hash table

☐

7. In **triplet representation**, how many columns are there?
- A. 1
 - B. 2
 - C. 3
 - D. 4

☐

8. In triplet representation, which of the following is stored?
- A. Only values
 - B. Only positions
 - C. Row, column, and value
 - D. Size of matrix

☐

9. What is the first row of a triplet representation of sparse matrix?
- A. Index row
 - B. Header row
 - C. Meta row
 - D. Result row

☐

10. The header row of triplet form stores:
- A. Only size
 - B. Row index

- C. Matrix dimensions and number of non-zero values
 - D. Only non-zero values
- ☐

☐ Operations on Sparse Matrices

11. Which operation is the **least efficient** on sparse matrices?
- A. Transpose
 - B. Addition
 - C. Multiplication
 - D. Indexing
- ☐
12. What is the main challenge in sparse matrix multiplication?
- A. Memory
 - B. Time
 - C. Alignment of non-zero elements
 - D. Compiler
- ☐
13. What is the time complexity of fast transpose of sparse matrix (triplet form with t non-zero values)?
- A. $O(n^2)$
 - B. $O(t^2)$
 - C. $O(t + n)$
 - D. $O(1)$
- ☐
14. What is required to efficiently **add two sparse matrices** in triplet form?
- A. Hash function
 - B. Matching row and column indices
 - C. Merge sort
 - D. Row-major order
- ☐
15. Sparse matrix multiplication is most efficient when:
- A. One matrix is dense
 - B. Both are sparse
 - C. Both are diagonal

D. One is upper triangular

☐

☐ Sparse Matrix in Practice

16. Which field uses sparse matrices **extensively**?

- A. Compiler design
- B. Operating systems
- C. Machine learning / NLP
- D. Microprocessor

☐

17. Which data structure is often used to store sparse matrices efficiently?

- A. Stack
- B. Queue
- C. Linked list
- D. Circular queue

☐

18. A common application of sparse matrices is:

- A. Page replacement
- B. Polynomial evaluation
- C. Graph adjacency matrices
- D. Parsing XML

☐

19. Which of the following statements is true?

- A. Sparse matrices waste memory
- B. Sparse matrices are always symmetric
- C. Sparse matrices are not used in practice
- D. Sparse matrices reduce memory usage

☐

20. Sparse matrices are better than arrays when:

- A. Many elements are zero
- B. Few elements are zero
- C. Elements are random
- D. It is a 1D structure

☐

□ CUET PG PYQs / PYQ-style

21. CUET PG-style: If a 4×4 matrix has 2 non-zero elements, how many rows will be in triplet form?

- A. 2
- B. 3
- C. 5
- D. 1

□

22. CUET PG-style: What is the space saved when storing a 5×5 sparse matrix with 3 non-zero elements using triplet format?

- A. $25 \rightarrow 9$
- B. $25 \rightarrow 12$
- C. $25 \rightarrow 3$
- D. $25 \rightarrow 6$

□

23. CUET PG-style: Which of the following is **not needed** in triplet representation?

- A. Column index
- B. Row index
- C. Matrix size
- D. Number of columns with zero

□

24. CUET PG-style: Transposing a sparse matrix involves:

- A. Flipping non-zero elements
- B. Interchanging row and column indices
- C. Replacing values
- D. Replacing matrix size

□

25. CUET PG-style: Which representation of sparse matrix is best suited for **row-wise traversal**?

- A. Triplet
- B. Dictionary
- C. Linked list
- D. Compressed row storage (CRS)

□

☐ **Advanced/Conceptual**

26. Compressed Sparse Row (CSR) uses how many arrays?

- A. 1
- B. 2
- C. 3
- D. 4

☐

27. In CSR, the first array stores:

- A. Column index
- B. Row pointer
- C. Values
- D. Matrix size

☐

28. A diagonal matrix is:

- A. Always sparse
- B. Never sparse
- C. Sometimes sparse
- D. Always dense

☐

29. Which one is **NOT** a reason to use sparse matrices?

- A. Faster access to all elements
- B. Memory efficiency
- C. Faster computation in special cases
- D. Compact storage

☐

30. A symmetric sparse matrix has:

- A. No diagonal
- B. Equal row and column values
- C. Mirror image entries
- D. Unequal non-zero values

☐

☐ **Answer Key:**

1-B 2-B 3-A 4-B 5-C

6-B 7-C 8-C 9-B 10-C

11-D 12-C 13-C 14-B 15-B

16-C 17-C 18-C 19-D 20-A

21-B 22-A 23-D 24-B 25-D

26-C 27-B 28-C 29-A 30-C

□ Topic: Stacks

Total MCQs: 30

Includes: Theory, Applications, Coding, CUET PYQs

Answer Key: At the end

□ Basics of Stack

1. A stack is a:
 - A. FIFO structure
 - B. LIFO structure
 - C. FILO structure
 - D. Both B and C☐
2. The operation that adds an element to the stack is called:
 - A. Push
 - B. Pop
 - C. Insert
 - D. Add☐
3. The operation that removes the top element of the stack is:
 - A. Peek
 - B. Pop
 - C. Push
 - D. Delete☐
4. What is the result of performing a pop on an empty stack?
 - A. Stack overflow

- B. Memory leak
- C. Stack underflow
- D. Segmentation fault

☐

5. Which of the following operations are fundamental to a stack?

- A. Push, Pull
- B. Add, Remove
- C. Push, Pop
- D. Insert, Delete

☐

☐ Stack Representation

6. Which data structure is used to implement a stack?

- A. Array
- B. Linked List
- C. Both A and B
- D. Queue

☐

7. What is the time complexity of push and pop operations in a stack?

- A. $O(n)$
- B. $O(\log n)$
- C. $O(1)$
- D. $O(n \log n)$

☐

8. Which of the following is not a valid stack operation?

- A. Push
- B. Top
- C. Insert at bottom
- D. Pop

☐

9. Which of the following data structures uses stack internally?

- A. Queue
- B. Binary Search Tree
- C. Recursion

D. Hash Table

☐

10. The top of the stack points to:

A. The bottom element

B. The latest inserted element

C. The oldest element

D. Null always

☐

☐ Applications of Stack

11. Stack is used in:

A. Backtracking

B. Expression evaluation

C. Recursion

D. All of the above

☐

12. Which algorithm uses stacks for backtracking?

A. Depth-First Search

B. Breadth-First Search

C. Binary Search

D. Bubble Sort

☐

13. What is the role of stack in function calls?

A. Stores return addresses

B. Stores arguments

C. Stores local variables

D. All of the above

☐

14. Which of the following expressions is valid postfix?

A. $A + B$

B. $+ A B$

C. $A B +$

D. $(A + B)$

☐

15. Which is used to convert infix to postfix?

- A. Kruskal's Algorithm
- B. Shunting Yard Algorithm
- C. DFS
- D. Bellman Ford Algorithm

☐

☐ Stack Implementation

16. What happens if we push an element onto a full stack (array implementation)?

- A. Stack underflow
- B. Stack overflow
- C. Segmentation fault
- D. Nothing

☐

17. Which of the following checks the top element without removing it?

- A. Pop
- B. Peek
- C. Top
- D. Both B and C

☐

18. What is the initial value of 'top' in an empty stack?

- A. 0
- B. 1
- C. -1
- D. NULL

☐

19. In a linked list-based stack, push operation:

- A. Adds node at head
- B. Adds node at tail
- C. Adds node in middle
- D. Reverses the list

☐

20. In stack, the maximum number of elements that can be stored is decided by:

- A. Stack Pointer
- B. Stack Size

C. Heap Size

D. OS



☐ **CUET PG Style / PYQ-Based**

21. CUET PG-style: After the following operations on a stack: push(1), push(2), pop(), push(3), pop(), what is on top?

A. 1

B. 2

C. 3

D. Empty



22. CUET PG-style: What is the postfix of the expression: $(A + B) * (C - D)$?

A. $AB + CD - *$

B. $A + B * C - D$

C. $AB + C - D *$

D. $ABCD +- *$



23. CUET PG-style: What is the output of the following stack sequence:

Push A, Push B, Pop, Push C, Pop, Pop

A. A B C

B. B C A

C. C B A

D. B C



24. CUET PYQ (2023): Which data structure uses **LIFO** order?

A. Queue

B. Stack

C. Deque

D. Tree



25. CUET PYQ: Which is used for **expression evaluation**?

A. Stack

B. Queue

C. Linked list

D. Hash table



☐ **Advanced / Mixed**

26. Which of the following is not an application of stack?

- A. Tower of Hanoi
- B. Syntax parsing
- C. Job scheduling
- D. Undo feature



27. Minimum number of stacks needed to sort a stack using another stack?

- A. 1
- B. 2
- C. 3
- D. 0



28. When is stack preferred over recursion?

- A. When memory is not an issue
- B. For simpler logic
- C. For iterative simulation of recursion
- D. Stack is never preferred



29. Which of the following can be used to implement multiple stacks in a single array?

- A. Divide array
- B. Dynamic arrays
- C. Linked list
- D. Hashing



30. Which technique is used to avoid stack overflow in recursion?

- A. Memoization
- B. Tail recursion
- C. Function pointers
- D. Depth-first traversal



□ Answer Key:

1-D 2-A 3-B 4-C 5-C

6-C 7-C 8-C 9-C 10-B

11-D 12-A 13-D 14-C 15-B

16-B 17-D 18-C 19-A 20-B

21-A 22-A 23-C 24-B 25-A

26-C 27-B 28-C 29-A 30-B

□ Topic: Queues

□ Basics of Queues

1. A queue is a:
 - A. LIFO structure
 - B. FIFO structure
 - C. FILO structure
 - D. Circular structure☐
2. The operation to insert an element into a queue is:
 - A. Pop
 - B. Push
 - C. Enqueue
 - D. Dequeue☐
3. The operation to remove an element from a queue is:
 - A. Enqueue
 - B. Pop
 - C. Dequeue
 - D. Peek☐
4. The time complexity of Enqueue and Dequeue operations is:
 - A. $O(1)$
 - B. $O(n)$

- C. $O(\log n)$
- D. $O(n \log n)$

☐

5. A queue is best implemented using:

- A. Array
- B. Linked list
- C. Stack
- D. Tree

☐

☐ Types of Queues

6. In which type of queue is the last position connected back to the first position?

- A. Double-ended queue
- B. Linear queue
- C. Circular queue
- D. Priority queue

☐

7. In a circular queue implemented using arrays, the condition for full queue is:

- A. $\text{rear} = \text{max} - 1$
- B. $(\text{rear} + 1) \% \text{size} == \text{front}$
- C. $\text{rear} == \text{front}$
- D. $\text{front} == -1$

☐

8. Double-ended queue allows insertion and deletion:

- A. Only at front
- B. Only at rear
- C. At both ends
- D. Only in middle

☐

9. Priority Queue is:

- A. FIFO
- B. LIFO
- C. Based on priority
- D. None of the above

☐

10. Which of the following queue allows elements to be inserted in any order but removed based on priority?
- A. Simple queue
 - B. Deque
 - C. Priority queue
 - D. Circular queue
- ☐
-

☐ Queue Operations & Implementation

11. What is the initial value of front and rear in an empty queue using arrays?
- A. 0
 - B. 1
 - C. -1
 - D. NULL
- ☐
12. What happens when you try to dequeue from an empty queue?
- A. Overflow
 - B. Underflow
 - C. Segmentation fault
 - D. Compilation error
- ☐
13. In a linear queue implemented using array, after a few dequeue operations, the front shifts but space at the beginning is:
- A. Automatically reused
 - B. Lost
 - C. Recycled
 - D. None of the above
- ☐
14. What is the disadvantage of a linear queue using arrays?
- A. Slow access
 - B. Overflow even if space is available
 - C. Expensive memory
 - D. Time complexity is $O(n^2)$
- ☐

15. What is the advantage of a circular queue over a linear queue?

- A. Easier implementation
- B. Saves memory
- C. Faster operations
- D. Priority support

☐

☐ Applications of Queues

16. Which of the following is **not** an application of queues?

- A. Printer queue
- B. Job scheduling
- C. Expression conversion
- D. Call center system

☐

17. Which queue is used for CPU scheduling?

- A. Priority queue
- B. Simple queue
- C. Circular queue
- D. None

☐

18. Which data structure is used in BFS (Breadth First Search)?

- A. Stack
- B. Queue
- C. Heap
- D. Tree

☐

19. In which scenario do circular queues help the most?

- A. Undo operation
- B. Browser history
- C. Round robin scheduling
- D. Recursion

☐

20. Which of the following operations is valid on a queue?

- A. Insert at front
- B. Delete from rear

- C. Insert at rear
 - D. Push at top
 - ☐
-

☐ **CUET PG Style / Previous Years-Based**

21. CUET PG-style: Which operation occurs at the rear of the queue?
- A. Deletion
 - B. Insertion
 - C. Traversal
 - D. Search
- ☐
22. CUET PG-style: A queue follows which order of insertion and deletion?
- A. LIFO
 - B. FIFO
 - C. Random
 - D. FILO
- ☐
23. CUET PYQ: What happens if we enqueue in a full queue?
- A. Overflow
 - B. Underflow
 - C. Stack full
 - D. Front error
- ☐
24. CUET PYQ: Queue used in Round Robin scheduling is:
- A. Priority queue
 - B. Circular queue
 - C. Stack
 - D. Array queue
- ☐
25. CUET PG-style: If rear = 5, front = 2 in a circular queue of size 6, how many elements are present?
- A. 4
 - B. 3
 - C. 5

D. 2

☐

☐ Advanced & Coding

26. What is the worst-case time complexity for searching an element in an unsorted queue?

- A. $O(1)$
- B. $O(\log n)$
- C. $O(n)$
- D. $O(n^2)$

☐

27. What will be the output of this operation: Enqueue(1), Enqueue(2), Dequeue(), Enqueue(3), Dequeue()?

- A. 2
- B. 1
- C. 3
- D. Queue is empty

☐

28. How many pointers are used in linked list implementation of a queue?

- A. 1
- B. 2
- C. 3
- D. 0

☐

29. In a circular queue of size n , how do you calculate next position of rear after insertion?

- A. $\text{rear} + 1$
- B. $\text{rear} + n$
- C. $(\text{rear} + 1) \% n$
- D. $(\text{rear} - 1) \% n$

☐

30. Which is true about a deque?

- A. Elements can be inserted only at one end
- B. It's a special type of stack
- C. Allows insertion and deletion at both ends
- D. Uses recursion

☐

□ Answer Key:

1-B 2-C 3-C 4-A 5-B

6-C 7-B 8-C 9-C 10-C

11-C 12-B 13-B 14-B 15-B

16-C 17-A 18-B 19-C 20-C

21-B 22-B 23-A 24-B 25-A

26-C 27-C 28-B 29-C 30-C

□ Topic: Priority Queues

Total Questions: 30

Includes: Basics, Implementation, Applications, Min/Max Heaps

Answer Key: At the end

□ Basics & Conceptual

1. What is a Priority Queue?
 - A. FIFO Queue
 - B. LIFO Queue
 - C. Queue where elements are ordered by priority
 - D. Stack with priority

□
2. In a priority queue, elements with:
 - A. Higher priority are dequeued first
 - B. Lower priority are dequeued first
 - C. Random priority is dequeued
 - D. Equal priority dequeued first

□
3. Which of the following data structures can be used to implement a priority queue efficiently?
 - A. Array
 - B. Stack

C. Linked List

D. Heap

☐

4. Which of the following supports priority-based insertion and removal?

A. Deque

B. Priority queue

C. Circular queue

D. Stack

☐

5. If two elements have the same priority in a priority queue, which one is served first?

A. LIFO

B. FIFO

C. Highest value

D. Random

☐

☐ Types & Heaps

6. Which heap is used in a min-priority queue?

A. Max Heap

B. Min Heap

C. Binary Search Tree

D. AVL Tree

☐

7. In a max-priority queue, which element is deleted first?

A. Smallest

B. Oldest

C. Largest

D. Middle

☐

8. A min-heap is:

A. Complete binary tree where parent < children

B. Binary search tree

C. AVL Tree

D. Random tree

☐

9. Which of the following is **true** for Max Heap?

- A. Root is smallest
- B. Root is largest
- C. Children are always smaller than siblings
- D. It's a linear structure

☐

10. Time complexity of inserting into a heap is:

- A. $O(1)$
- B. $O(n)$
- C. $O(\log n)$
- D. $O(n \log n)$

☐

☐ Operations

11. What is the time complexity of deleting the highest priority element in a heap-based priority queue?

- A. $O(1)$
- B. $O(\log n)$
- C. $O(n)$
- D. $O(n^2)$

☐

12. Inserting into an unsorted array-based priority queue takes:

- A. $O(1)$
- B. $O(n)$
- C. $O(\log n)$
- D. $O(n \log n)$

☐

13. Removing from a sorted array-based priority queue takes:

- A. $O(n)$
- B. $O(\log n)$
- C. $O(1)$
- D. $O(n^2)$

☐

14. Which operation can violate the heap order property?

- A. Access

- B. Delete
- C. Insertion
- D. Traverse

☐

15. Heapify operation is used in:

- A. Stack
- B. Queue
- C. Priority queue
- D. Hash Table

☐

☐ Implementation & Behavior

16. Which of these can be used to represent a binary heap?

- A. Hash Table
- B. Linked list
- C. Array
- D. Tree

☐

17. Index of left child in an array-based binary heap at index i is:

- A. $2i$
- B. $2i + 1$
- C. $(i-1)/2$
- D. $i+1$

☐

18. Index of right child in an array-based binary heap at index i is:

- A. $2i + 1$
- B. $2i$
- C. $i-1$
- D. $i+2$

☐

19. What is the parent index of a node at index i in array heap representation?

- A. $i/2$
- B. $(i-1)/2$
- C. $2*i$

D. $i+1$

☐

20. Binary heap is a:

A. Complete binary tree

B. AVL Tree

C. Full binary tree

D. Random tree

☐

☐ **CUET PG Pattern / PYQ-style**

21. CUET PYQ-style: What is the worst-case time to delete the root of a max heap with n elements?

A. $O(1)$

B. $O(n)$

C. $O(\log n)$

D. $O(n \log n)$

☐

22. CUET PG-style: Priority queues are mainly used in:

A. DFS

B. Dijkstra's algorithm

C. Linear search

D. Infix expression evaluation

☐

23. CUET PG-style: What will be the order of removal from a min-priority queue with values: $\{5, 2, 8, 1\}$?

A. 5 2 8 1

B. 1 2 5 8

C. 8 5 2 1

D. Random

☐

24. CUET PYQ-style: Which data structure is most efficient for priority queue implementation?

A. Array

B. Heap

C. Tree

D. Stack

☐

25. CUET-style: When inserting into a min heap, where is the element first inserted?

A. At the root

B. At the last level, leftmost spot

C. Anywhere

D. At middle

☐

☐ Applications

26. Which of these is a major real-world application of priority queues?

A. Stack memory

B. Graph traversal

C. Task scheduling

D. Undo functionality

☐

27. In OS, priority queues are used for:

A. File management

B. Process scheduling

C. Memory allocation

D. Disk cleanup

☐

28. Which algorithm uses a priority queue to find the shortest path?

A. DFS

B. BFS

C. Dijkstra's

D. Kruskal's

☐

29. In which type of queue are the elements dequeued according to their importance?

A. Circular queue

B. Linear queue

C. Deque

D. Priority queue

☐

30. Best data structure to implement a job scheduler is:

- A. Stack
- B. Queue
- C. Priority queue
- D. Hash map

□

□ **Answer Key:**

1-C 2-A 3-D 4-B 5-B

6-B 7-C 8-A 9-B 10-C

11-B 12-A 13-C 14-C 15-C

16-C 17-A 18-A 19-B 20-A

21-C 22-B 23-B 24-B 25-B

26-C 27-B 28-C 29-D 30-C

□ **Topic: Linked Lists**

Total Questions: 30

Covers: Types, Operations, Applications, Pointer Behavior

Answer Key: At the end

CUET PYQs Included

□ **Basics and Types**

1. A linked list is a:

- A. Linear data structure
- B. Non-linear data structure
- C. Primitive data type
- D. Array-based structure

□

2. What does each node in a singly linked list contain?

- A. Only data
- B. Data and address of next node

C. Data and address of previous node

D. Only pointer

☐

3. In a doubly linked list, each node contains:

A. One pointer and data

B. Two pointers and data

C. Three pointers

D. No data

☐

4. In circular linked list:

A. Last node points to NULL

B. First node points to last node

C. Last node points to first node

D. List is infinite

☐

5. Which of the following is FALSE about linked lists?

A. Can grow dynamically

B. Faster access to middle element

C. Used for memory-efficient insertions/deletions

D. Can be singly or doubly linked

☐

☐ Operations

6. Time complexity of insertion at the beginning in singly linked list:

A. $O(n)$

B. $O(\log n)$

C. $O(1)$

D. $O(n \log n)$

☐

7. Time complexity of searching an element in a singly linked list is:

A. $O(1)$

B. $O(\log n)$

C. $O(n)$

D. $O(n^2)$

☐

8. Deletion from the end of singly linked list (without tail pointer) requires:

- A. $O(1)$
- B. $O(n)$
- C. $O(n^2)$
- D. $O(\log n)$

☐

9. In a doubly linked list, deletion at any position is:

- A. $O(n)$
- B. $O(1)$
- C. $O(n \log n)$
- D. $O(n^2)$

☐

10. Which pointer is modified while inserting a node at the front?

- A. Tail only
- B. Head only
- C. Head and new node
- D. None

☐

☐ Advanced Concepts

11. Which data structure is best for implementing undo features?

- A. Stack
- B. Queue
- C. Doubly linked list
- D. Circular queue

☐

12. Which of the following is used for memory-efficient representation of a linked list?

- A. Hashing
- B. XOR linked list
- C. Stack
- D. Tree

☐

13. In XOR linked list, each node stores:

- A. Only data
- B. XOR of next and previous node addresses

C. XOR of current and head

D. Both A and B

☐

14. Which linked list doesn't contain NULL?

A. Doubly

B. Circular

C. Singly

D. None

☐

15. Which is true about singly linked list?

A. Allows backward traversal

B. Needs extra memory per node

C. Insertion in middle is $O(1)$

D. Next pointer of last node is NULL

☐

☐ CUET PG-style & PYQ

16. CUET PG PYQ-style: Which of these is used to implement a polynomial?

A. Array

B. Queue

C. Linked list

D. Hash map

☐

17. CUET PG-style: Linked list is preferred over array when:

A. Random access is required

B. Frequent insertions/deletions required

C. Memory size is fixed

D. Search is frequent

☐

18. CUET PYQ: What will be time complexity to reverse a singly linked list of n nodes?

A. $O(1)$

B. $O(n)$

C. $O(n \log n)$

D. $O(n^2)$

☐

19. CUET-style: Which traversal is not possible in singly linked list?

- A. From head to tail
- B. From tail to head
- C. Full traversal
- D. All are possible

☐

20. CUET PG-style: Which pointer in DLL helps in reverse traversal?

- A. prev
- B. next
- C. head
- D. tail

☐

☐ Code-Based & Pointer Logic

21. If head points to a node and you do `head = head->next;`, what happens?

- A. First node is deleted
- B. List is reversed
- C. Head moves to next node
- D. Error

☐

22. To delete the head node in singly linked list:

- A. `Head = head->next`
- B. `Head = head + 1`
- C. `Head = NULL`
- D. `Head->next = NULL`

☐

23. Which operation needs full traversal in singly linked list?

- A. Insertion at front
- B. Insertion at end
- C. Insertion at head
- D. Deletion at head

☐

24. Which of these is best for implementation of a queue?

- A. Array
- B. Stack

- C. Singly linked list
- D. Doubly linked list

☐

25. Which node is used to detect loop in a linked list using Floyd's algorithm?

- A. Front node
- B. Two pointers (slow and fast)
- C. Only tail
- D. None

☐

☐ Applications and Real Use

26. Which of these can be implemented using linked list?

- A. Stack
- B. Queue
- C. Graph
- D. All of the above

☐

27. Which of the following is **NOT** an application of linked lists?

- A. Memory management
- B. Implementing browser history
- C. Real-time CPU scheduling
- D. Random access operations

☐

28. Which of these is a disadvantage of linked lists over arrays?

- A. Slower insertions
- B. Extra space for pointers
- C. Fixed size
- D. High memory requirement

☐

29. A node in linked list contains how many fields (in DLL)?

- A. 1
- B. 2
- C. 3
- D. 4

☐

30. In linked lists, memory is allocated:

- A. Statically
- B. Contiguously
- C. Dynamically
- D. Randomly

□

□ Answer Key

1-A 2-B 3-B 4-C 5-B

6-C 7-C 8-B 9-B 10-C

11-C 12-B 13-B 14-B 15-D

16-C 17-B 18-B 19-B 20-A

21-C 22-A 23-B 24-D 25-B

26-D 27-D 28-B 29-C 30-C

□ Topic: Trees

Total MCQs: 30

Covers: Tree Terminology, Binary Trees, Traversals, BST, AVL Trees, Heaps, Applications

CUET PYQs: Marked □

Answer Key: At the end

□ Basics & Terminology

1. A tree is a:

- A. Linear data structure
- B. Non-linear data structure
- C. Sequential structure
- D. None of the above

□

2. The number of edges in a tree with n nodes is:

- A. n
- B. n-1

C. $n+1$

D. $2n$

☐

3. Which of the following is true for a tree?

A. No cycles

B. Exactly one root

C. All nodes except root have one parent

D. All of the above

☐

4. The height of a tree is:

A. Number of leaves

B. Number of nodes

C. Length of the longest path from root to a leaf

D. Number of internal nodes

☐

5. The maximum number of nodes at level l in a binary tree is:

A. $2^{(l-1)}$

B. l

C. $2l$

D. l^2

☐

☐ Binary Trees & Binary Search Trees (BST)

6. In a Binary Tree, a node can have at most:

A. 1 child

B. 2 children

C. 3 children

D. 4 children

☐

7. Binary Search Tree (BST) stores elements in such a way that:

A. $\text{Left} < \text{Root} < \text{Right}$

B. $\text{Left} > \text{Root} > \text{Right}$

C. $\text{Root} < \text{Left} < \text{Right}$

D. $\text{Root} > \text{Left} > \text{Right}$

☐

8. In a BST, the inorder traversal gives:
- A. Preorder list
 - B. Sorted list
 - C. Postorder list
 - D. Reverse sorted list
- ☐
9. Time complexity to search an element in a balanced BST (with n nodes) is:
- A. $O(n)$
 - B. $O(1)$
 - C. $O(\log n)$
 - D. $O(n \log n)$
- ☐
10. Which traversal of a BST outputs data in descending order?
- A. Inorder
 - B. Preorder
 - C. Postorder
 - D. Reverse Inorder
- ☐
-

☐ Tree Traversals

11. Preorder traversal sequence is:
- A. Left $\rightarrow$ Right $\rightarrow$ Root
 - B. Root $\rightarrow$ Left $\rightarrow$ Right
 - C. Right $\rightarrow$ Left $\rightarrow$ Root
 - D. Left $\rightarrow$ Root $\rightarrow$ Right
- ☐
12. Postorder traversal of a binary tree visits:
- A. Root $\rightarrow$ Left $\rightarrow$ Right
 - B. Left $\rightarrow$ Right $\rightarrow$ Root
 - C. Right $\rightarrow$ Root $\rightarrow$ Left
 - D. None
- ☐
13. Level order traversal is implemented using:
- A. Stack
 - B. Recursion

- C. Queue
- D. Hash Table
- ☐

14. Which traversal is best suited for copying a tree?

- A. Preorder
- B. Inorder
- C. Postorder
- D. Level-order
- ☐

15. Which traversal is used to delete the tree?

- A. Inorder
 - B. Preorder
 - C. Postorder
 - D. Level order
 - ☐
-

☐ Heaps, AVL, and Balanced Trees

16. In a min-heap:

- A. Parent is always smaller than child
- B. Parent is always greater than child
- C. Heap is sorted
- D. All levels are filled
- ☐

17. Which is true for AVL trees?

- A. They are always balanced
- B. Balance factor is -1, 0, or +1
- C. Search time is $O(\log n)$
- D. All of the above
- ☐

18. Balance factor in AVL tree =

- A. Height of left subtree - right subtree
- B. Height of root
- C. Total number of nodes
- D. Difference between leaf nodes
- ☐

19. What is the max height of AVL tree with 7 nodes?

- A. 2
- B. 3
- C. 4
- D. 5

☐

20. Max-heap is typically used in:

- A. Searching
- B. Heapsort
- C. DFS
- D. BFS

☐

☐ CUET PG PYQs & CUET-style Questions

21. ☐CUET PYQ: Which traversal is used to evaluate postfix expression in an expression tree?

- A. Inorder
- B. Preorder
- C. Postorder
- D. Level order

☐

22. ☐CUET PYQ: Which is the correct property of a binary search tree?

- A. Left subtree > root
- B. Left subtree < root < right subtree
- C. All elements same
- D. No parent-child relationship

☐

23. ☐CUET-style: If a node has 2 children, in a BST deletion, which node can replace it?

- A. Root
- B. Any leaf node
- C. Inorder successor or predecessor
- D. Its sibling

☐

24. CUET-style: Which of the following is NOT a balanced tree?

- A. AVL
- B. Red-Black Tree

- C. Binary Heap
- D. Unsorted Binary Tree

☐

25. CUET-style: Which tree gives $O(1)$ time for finding minimum?

- A. Max heap
- B. Min heap
- C. Binary tree
- D. BST

☐

☐ Applications & Problem Solving

26. Trees are used in:

- A. File systems
- B. Routing algorithms
- C. Compiler syntax trees
- D. All of the above

☐

27. Which tree is used for maintaining priority in job scheduling?

- A. BST
- B. Heap
- C. AVL
- D. Red-Black Tree

☐

28. Which is not a type of binary tree?

- A. Strict binary tree
- B. Complete binary tree
- C. Skewed tree
- D. Circular binary tree

☐

29. Minimum number of nodes in a complete binary tree of height h is:

- A. $2^h - 1$
- B. $2^{(h-1)}$
- C. h
- D. $h+1$

☐

30. Number of leaf nodes in a full binary tree with n internal nodes:

- A. $n-1$
 - B. n
 - C. $n+1$
 - D. $2n$
- ☐

☐ **Answer Key**

1-B 2-B 3-D 4-C 5-A

6-B 7-A 8-B 9-C 10-D

11-B 12-B 13-C 14-A 15-C

16-A 17-D 18-A 19-C 20-B

21-C 22-B 23-C 24-D 25-B

26-D 27-B 28-D 29-C 30-C

☐ **Topic: Forest in Data Structures**

Total MCQs: 30

Includes: Basic concepts, tree-to-forest conversion, forest-to-tree conversion, applications

PYQs: Marked ☐

Answer key: At the end

☐ **Basics of Forest**

1. A forest is:

- A. A collection of binary trees
 - B. A collection of disjoint trees
 - C. A circular list of trees
 - D. A single tree
- ☐

2. A forest can be converted into:

- A. Linked list
- B. Heap

C. Binary tree

D. Array

☐

3. The number of trees in a forest with n nodes and e edges is:

A. n

B. e

C. $n - e$

D. $e - n$

☐

4. Which of the following is **not** a forest?

A. Set of disjoint acyclic graphs

B. Collection of trees

C. Graph with cycles

D. Set of multiple trees

☐

5. A tree is a special case of a:

A. Forest

B. Graph

C. Binary Tree

D. Heap

☐

☐ Tree to Forest Conversion

6. Removing the root of a tree results in:

A. A heap

B. A forest

C. A queue

D. An empty tree

☐

7. A forest is obtained by:

A. Removing leaf nodes

B. Removing internal nodes

C. Removing the root of a tree

D. Replacing all edges

☐

8. If a tree has 3 subtrees and its root is removed, how many trees are in the resulting forest?
- A. 2
 - B. 3
 - C. 1
 - D. 0
- ☐
9. Which traversal is best suited to convert a tree to a forest?
- A. Postorder
 - B. Preorder
 - C. Inorder
 - D. Level-order
- ☐
10. Number of edges in a forest with k trees and n total nodes =
- A. $n + k$
 - B. $n - k$
 - C. $k - n$
 - D. $2n$
- ☐
-

☐ Forest to Binary Tree Conversion

11. Forest can be converted into binary tree using:
- A. Right child, right sibling method
 - B. Left child, right sibling method
 - C. Left child, left sibling method
 - D. Parent pointer method
- ☐
12. In the **Left Child Right Sibling (LCRS)** representation:
- A. Only left pointers are used
 - B. Each node has two pointers
 - C. Each node has one child
 - D. The tree becomes a heap
- ☐
13. In LCRS, what does the right pointer point to?
- A. Parent

- B. Next sibling
- C. Root
- D. NULL

☐

14. In LCRS representation, left pointer represents:

- A. Leftmost child
- B. Right sibling
- C. Parent
- D. Leaf node

☐

15. The maximum number of binary trees that can form a forest with 4 nodes:

- A. 4
- B. 8
- C. 14
- D. 16

☐

☐ CUET PG PYQs & CUET-style

16. ☐ CUET-style: In a forest with 7 nodes and 4 edges, the number of trees is:

- A. 2
- B. 3
- C. 4
- D. 5

☐

17. ☐ CUET-style: A forest has 3 trees and 10 nodes. How many edges are there?

- A. 7
- B. 8
- C. 9
- D. 10

☐

18. ☐ CUET-style: What data structure is used for LCRS representation?

- A. Array
- B. Binary tree
- C. Heap

D. Queue

☐

19. ☐ CUET-style: When a root of a tree is deleted, the result is:

A. A linked list

B. A heap

C. A forest

D. A graph

☐

20. ☐ CUET-style: In a forest, each connected component is:

A. A heap

B. A graph

C. A tree

D. A linked list

☐

☐ Applications & Problem Solving

21. Forests are used in:

A. File directories

B. Symbol tables

C. Network hierarchy

D. All of the above

☐

22. Which of the following operations can break a tree into forest?

A. Deletion of root

B. Deletion of any internal node

C. Deletion of edges

D. All of the above

☐

23. The height of a forest is:

A. Height of tallest tree

B. Sum of all tree heights

C. Average tree height

D. Always zero

☐

24. LCRS conversion helps in:

- A. Memory saving
- B. Efficient search
- C. Representation of general trees
- D. Both A and C

☐

25. If each node has at most k children, then forest is called:

- A. General tree
- B. K -ary forest
- C. Binary forest
- D. AVL forest

☐

☐ Advanced Concepts

26. Which of the following is true for a forest with n nodes and k trees?

- A. It has $n + k$ edges
- B. It has $n - k$ edges
- C. It has $n \times k$ edges
- D. None of these

☐

27. A general tree can be represented using:

- A. Array
- B. Stack
- C. Queue
- D. Binary tree

☐

28. In LCRS, traversal order equivalent to **postorder** of forest is:

- A. Inorder of LCRS tree
- B. Postorder of LCRS tree
- C. Level-order of LCRS
- D. Preorder of LCRS

☐

29. What is the major benefit of using forests?

- A. Reduces complexity
- B. Dynamic addition of trees

C. Easier hierarchy modeling

D. All of the above

☐

30. Which is true about forest traversal?

A. Each tree is traversed independently

B. Forest can't be traversed

C. Always in preorder

D. Only via recursion

☐

☐ **Answer Key:**

1-B 2-C 3-C 4-C 5-A

6-B 7-C 8-B 9-A 10-B

11-B 12-B 13-B 14-A 15-D

16-D 17-A 18-B 19-C 20-C

21-D 22-D 23-A 24-D 25-B

26-B 27-D 28-B 29-D 30-A

☐ **Topic: Binary Tree**

Total MCQs: 30

Covers: Basics, properties, traversals, types, construction, and applications

Includes: ☐ CUET PYQs & model questions

Answer key: At the end

☐ **Basics & Properties**

1. A binary tree is a tree in which each node has at most:

A. One child

B. Two children

C. Three children

D. Any number of children

☐

2. In a binary tree of n nodes, how many null pointers are there?
- A. n
 - B. $n + 1$
 - C. $n - 1$
 - D. $2n$
- ☐
3. Maximum number of nodes in a binary tree of height h is:
- A. 2^h
 - B. $2^h - 1$
 - C. h
 - D. $h - 1$
- ☐
4. Maximum number of nodes at level l of a binary tree is:
- A. 2^l
 - B. l^2
 - C. l
 - D. $2l - 1$
- ☐
5. The number of internal nodes in a full binary tree with n leaf nodes is:
- A. n
 - B. $n - 1$
 - C. $n + 1$
 - D. $2n - 1$
- ☐
-

☐ Tree Traversals

6. Inorder traversal of a binary search tree (BST) gives:
- A. Descending order
 - B. Preorder
 - C. Sorted order
 - D. Postorder
- ☐
7. The postorder traversal of a binary tree is:
- A. Left, Right, Root
 - B. Root, Left, Right

C. Right, Root, Left

D. Left, Root, Right

☐

8. Preorder traversal sequence:

A. Root, Left, Right

B. Left, Root, Right

C. Left, Right, Root

D. Root, Right, Left

☐

9. Which traversal is used to evaluate expressions?

A. Inorder

B. Preorder

C. Postorder

D. Level order

☐

10. Which traversal uses a queue?

A. Inorder

B. Preorder

C. Postorder

D. Level order

☐

☐ Construction & Representation

11. A binary tree can be uniquely constructed from:

A. Preorder and Inorder

B. Postorder and Preorder

C. Level-order and Inorder

D. Inorder only

☐

12. A binary tree can be stored in an array using which traversal?

A. Postorder

B. Inorder

C. Level-order

D. Preorder

☐

13. Number of binary trees with n nodes is given by:

- A. $n!$
- B. Catalan number
- C. Fibonacci number
- D. n^2

☐

14. In binary tree representation using array, for node at index i , left child index is:

- A. $i+1$
- B. $2i$
- C. $2i+1$
- D. $2i-1$

☐

15. In array representation, parent of node at index i is at:

- A. $i/2$
- B. $(i-1)/2$
- C. $i-1$
- D. $2*i$

☐

☐ Types of Binary Trees

16. A **complete binary tree** is:

- A. All levels are completely filled
- B. All nodes have 2 children
- C. All levels except possibly the last are full
- D. Only root exists

☐

17. A **full binary tree** has:

- A. All internal nodes with two children
- B. All leaves at same level
- C. All nodes with at most one child
- D. Balanced height

☐

18. A **perfect binary tree** is:

- A. All levels full
- B. Only one leaf

- C. All internal nodes have one child
 - D. Random structure
- ☐

19. A **skewed binary tree** is:

- A. Height = $\log n$
 - B. All nodes have only left or only right child
 - C. Perfectly balanced
 - D. Tree with even number of nodes
- ☐

20. Maximum height of a skewed binary tree with n nodes is:

- A. $\log n$
 - B. n
 - C. $n/2$
 - D. $\sqrt{n}$
- ☐
-

☐ CUET PYQs & Model Questions ☐

21. ☐ CUET-style: The inorder traversal of a BST is 10, 20, 30, 40, 50. The root is:

- A. 10
 - B. 20
 - C. 30
 - D. 40
- ☐

22. ☐ CUET PYQ: If a tree has n leaf nodes, the number of internal nodes in full binary tree is:

- A. n
 - B. $n-1$
 - C. $n+1$
 - D. $n/2$
- ☐

23. ☐ CUET-style: Which data structure is used in level-order traversal?

- A. Stack
 - B. Queue
 - C. Array
 - D. Tree
- ☐

24. ☐ CUET-style: Which of the following traversals is used for copying a tree?
- A. Inorder
 - B. Postorder
 - C. Preorder
 - D. Level order
- ☐
25. ☐ CUET-style: What is the time complexity of searching in a balanced binary search tree?
- A. $O(n)$
 - B. $O(n \log n)$
 - C. $O(\log n)$
 - D. $O(1)$
- ☐
-

☐ Advanced Concepts

26. Which tree traversal is non-recursive using stack?
- A. Level-order
 - B. Preorder
 - C. Postorder
 - D. Inorder
- ☐
27. Time complexity of inorder traversal of a binary tree with n nodes is:
- A. $O(1)$
 - B. $O(\log n)$
 - C. $O(n)$
 - D. $O(n \log n)$
- ☐
28. If a binary tree has height h , then its minimum number of nodes is:
- A. h
 - B. $h + 1$
 - C. $h - 1$
 - D. $h + 2$
- ☐
29. Maximum number of leaves in a binary tree of height h is:
- A. 2^h
 - B. h

C. $2^{(h-1)}$

D. h^2

☐

30. In a BST, all values in left subtree are:

A. Smaller than root

B. Larger than root

C. Equal to root

D. Random

☐

☐ **Answer Key:**

1-B 2-D 3-B 4-A 5-B

6-C 7-A 8-A 9-C 10-D

11-A 12-C 13-B 14-C 15-B

16-C 17-A 18-A 19-B 20-B

21-C 22-B 23-B 24-C 25-C

26-D 27-C 28-B 29-C 30-A

☐ **Topic: Threaded Binary Tree**

Total Questions: 30

Includes: Concepts, advantages, traversal, and previous year-style CUET questions

Answer key: At the end

☐ **Basics of Threaded Binary Trees**

1. In a threaded binary tree, NULL pointers are replaced with:

A. Zero

B. NULL again

C. Threads

D. Memory addresses

☐

2. The main purpose of a threaded binary tree is to:
- A. Reduce memory
 - B. Perform faster searching
 - C. Perform faster traversal
 - D. Balance the tree
- ☐
3. In a **right threaded binary tree**, right null pointers point to:
- A. Right child
 - B. Inorder predecessor
 - C. Inorder successor
 - D. NULL
- ☐
4. In a **left threaded binary tree**, left null pointers point to:
- A. Left child
 - B. Inorder predecessor
 - C. Inorder successor
 - D. NULL
- ☐
5. In a **fully threaded binary tree**, both NULLs in each node point to:
- A. NULL
 - B. Predecessor and successor
 - C. Parent
 - D. Left child only
- ☐
-

☐ Structure and Types

6. A node in a threaded binary tree contains how many link fields (typically)?
- A. 2
 - B. 3
 - C. 4
 - D. 1
- ☐
7. Threaded binary trees help in traversing the tree without using:
- A. Queue
 - B. Stack or Recursion

- C. Heap
- D. Linked List
- ☐

8. Inorder traversal using threads avoids the use of:

- A. Recursion
- B. Stack
- C. Both A and B
- D. Loop
- ☐

9. A right thread of a node points to its:

- A. Right child
- B. Inorder successor
- C. Inorder predecessor
- D. Parent
- ☐

10. A left thread of a node points to its:

- A. Left child
 - B. Inorder successor
 - C. Inorder predecessor
 - D. Root
 - ☐
-

☐ Traversal and Operations

11. The main advantage of using a threaded binary tree is:

- A. Faster deletion
- B. Memory optimization
- C. Faster inorder traversal
- D. Easier insertion
- ☐

12. Which traversal benefits the most from threading?

- A. Inorder
- B. Preorder
- C. Postorder
- D. Level order
- ☐

13. Time complexity of inorder traversal in threaded binary tree is:

- A. $O(\log n)$
- B. $O(n \log n)$
- C. $O(n)$
- D. $O(1)$

☐

14. Threads are added to:

- A. Reduce depth
- B. Replace missing children
- C. Link nodes in traversal order
- D. Maintain height

☐

15. Which bit indicates whether a pointer is a thread or a child link?

- A. Link flag
- B. Balance factor
- C. Thread bit
- D. Heap bit

☐

☐ Construction and Applications

16. What is the main memory advantage of threaded binary trees?

- A. Compact representation
- B. Heap allocation
- C. Pre-allocated nodes
- D. Use of arrays

☐

17. Which traversal uses successor threads efficiently?

- A. Inorder
- B. Postorder
- C. Preorder
- D. Random

☐

18. Threaded binary tree eliminates:

- A. Stack
- B. Recursion

C. Both A and B

D. None

☐

19. To insert a node in a threaded binary tree, we must:

A. Balance tree

B. Remove all threads

C. Adjust threads and pointers

D. None of the above

☐

20. A fully threaded binary tree has:

A. One thread per node

B. All null pointers replaced

C. No threads

D. Balanced structure

☐

☐ CUET PYQ Style and Advanced

21. ☐ CUET-style: In a threaded binary tree, which of the following is not used?

A. Stack

B. Threads

C. Recursion

D. None

☐

22. ☐ CUET-style: Which pointer in a threaded binary tree may represent a thread?

A. Only left

B. Only right

C. Both left and right

D. None

☐

23. ☐ CUET-style: In inorder threaded binary tree, to find the successor of a node:

A. Move to left child

B. Use stack

C. Follow right thread

D. Use recursion

☐

24. ☐ CUET-style: Which of the following is true about threaded binary trees?

- A. Makes searching faster
- B. Uses heap
- C. Makes traversal faster without stack
- D. Always balanced

☐

25. ☐ CUET-style: The threading technique is most efficient for:

- A. Balanced trees
- B. Dense trees
- C. Skewed trees
- D. Trees where traversal is frequent

☐

☐ Theoretical and Conceptual

26. The predecessor in a threaded binary tree can be found via:

- A. Left pointer
- B. Right pointer
- C. Thread
- D. Parent link

☐

27. Threading helps avoid:

- A. Infinite loops
- B. Stack overflow
- C. Height imbalance
- D. Heap memory issues

☐

28. In right-inorder threading, the right pointer of a leaf node is used to point to:

- A. Right child
- B. Parent
- C. Inorder successor
- D. NULL

☐

29. The extra bit in each node of a threaded binary tree is used to:

- A. Indicate balance
- B. Store address

- C. Differentiate between child and thread
- D. Store parent
- ☐

30. The threaded binary tree is a variation of:

- A. BST
 - B. AVL
 - C. Red-Black Tree
 - D. Ternary Tree
 - ☐
-

☐ Answer Key

1-C 2-C 3-C 4-B 5-B

6-B 7-B 8-C 9-B 10-C

11-C 12-A 13-C 14-C 15-C

16-A 17-A 18-C 19-C 20-B

21-D 22-C 23-C 24-C 25-D

26-C 27-B 28-C 29-C 30-A

☐ Topic: Binary Search Tree (BST)

Total Questions: 30

Includes: Theory, traversal, insertion/deletion, applications, and PYQ-style

Answer key: At the end

☐ Basics of BST

1. A Binary Search Tree (BST) is a binary tree where:
 - A. Left subtree has greater values than node
 - B. Right subtree has smaller values
 - C. Left subtree has smaller and right subtree has greater values
 - D. No ordering rules
 - ☐

2. In a BST, the left child of a node contains:
- A. Smaller or equal key
 - B. Always smaller key
 - C. Larger key
 - D. None of the above
- ☐
3. In a BST, which traversal gives nodes in sorted order?
- A. Preorder
 - B. Inorder
 - C. Postorder
 - D. Level order
- ☐
4. Which of the following is true for BST?
- A. Duplicates are allowed always
 - B. Duplicates are never allowed
 - C. Depends on implementation
 - D. All duplicates go to right
- ☐
5. Which operation is fastest in a balanced BST?
- A. Insertion
 - B. Deletion
 - C. Search
 - D. Traversal
- ☐
-

☐ **BST Operations**

6. Time complexity for searching in a BST (average case):
- A. $O(1)$
 - B. $O(\log n)$
 - C. $O(n)$
 - D. $O(n \log n)$
- ☐
7. Worst-case time complexity for BST search occurs when tree is:
- A. Full
 - B. Balanced

- C. Skewed
- D. Complete

☐

8. In a BST, insertion of a new node occurs:

- A. At root
- B. At leaf position
- C. At internal node
- D. Random

☐

9. Deletion of a node with two children in a BST involves:

- A. Removing left child
- B. Replacing with root
- C. Replacing with inorder successor or predecessor
- D. Deleting whole subtree

☐

10. Which node is replaced during deletion of node with two children?

- A. Leftmost of left subtree
- B. Rightmost of left subtree
- C. Leftmost of right subtree
- D. Rightmost of right subtree

☐

☐ Traversals

11. Inorder traversal of BST:

- A. Preorder
- B. Sorted order
- C. Reverse order
- D. Depends on height

☐

12. Preorder traversal of BST is used for:

- A. Balanced tree generation
- B. Sorting
- C. Copying tree
- D. Deletion

☐

13. Postorder traversal is useful for:

- A. Inserting elements
- B. Tree deletion
- C. Search
- D. Tree height

☐

14. What is output of inorder traversal of BST with elements: 10, 20, 5, 15?

- A. 10, 20, 5, 15
- B. 5, 10, 15, 20
- C. 20, 15, 10, 5
- D. 10, 5, 20, 15

☐

15. If you perform inorder on BST and get 1, 2, 3, 4, 5, the tree is:

- A. Skewed left
- B. Balanced
- C. Skewed right
- D. Random

☐

☐ **BST Theory and Properties**

16. Which of the following BSTs is height-balanced?

- A. Complete tree
- B. AVL Tree
- C. B-Tree
- D. Any tree

☐

17. A skewed BST (right-heavy) behaves like:

- A. Queue
- B. Stack
- C. Linked List
- D. Heap

☐

18. Number of comparisons in worst-case search of BST is:

- A. $O(1)$
- B. $O(\log n)$

- C. $O(n)$
- D. $O(n^2)$
- ☐

19. BSTs are not efficient when:

- A. Balanced
- B. Sorted input used
- C. Search operations dominate
- D. Nodes are few
- ☐

20. When inserting 50 into a BST with root 30 and right child 40, 50 will be:

- A. Left of 30
 - B. Right of 30
 - C. Right of 40
 - D. Left of 40
 - ☐
-

☐ CUET PG Style PYQs

21. ☐ CUET-style: Which traversal gives reverse sorted elements in BST?

- A. Inorder
- B. Preorder
- C. Postorder
- D. Reverse Inorder
- ☐

22. ☐ CUET-style: Deleting root with two children, we replace it with:

- A. Any leaf node
- B. Random node
- C. Inorder predecessor or successor
- D. Left child
- ☐

23. ☐ CUET-style: Time complexity of inserting in BST (average):

- A. $O(1)$
- B. $O(\log n)$
- C. $O(n)$
- D. $O(n^2)$
- ☐

24. ☐ CUET-style: In BST, inorder traversal of [25, 15, 50, 10, 22] is:

- A. 10, 15, 22, 25, 50
- B. 25, 15, 50, 10, 22
- C. 50, 25, 22, 15, 10
- D. None

☐

25. ☐ CUET-style: BSTs are useful in:

- A. Memory management
- B. Searching and sorting
- C. Compilers
- D. Networks

☐

☐ Advanced and Application

26. BSTs are used to implement:

- A. Stacks
- B. Queues
- C. Sets and Maps
- D. Graphs

☐

27. Which of the following can degenerate a BST to linear time?

- A. Balanced data
- B. Random input
- C. Sorted input
- D. Repeated deletions

☐

28. Which BST operation is recursive in nature?

- A. Search
- B. Insertion
- C. Deletion
- D. All of the above

☐

29. Balanced BSTs like AVL/Red-Black Tree improve:

- A. Time complexity
- B. Memory

- C. Tree height
- D. Pointer usage

☐

30. Number of nodes in perfect BST of height h:

- A. 2^h
- B. $2^h - 1$
- C. h
- D. h^2

☐

☐ Answer Key

1-C 2-B 3-B 4-C 5-C

6-B 7-C 8-B 9-C 10-C

11-B 12-C 13-B 14-B 15-C

16-B 17-C 18-C 19-B 20-C

21-D 22-C 23-B 24-A 25-B

26-C 27-C 28-D 29-A 30-B

☐ Topic: AVL Tree (Adelson-Velsky and Landis Tree)

Total Questions: 30

Covers: Rotations, Insertion/Deletion, Balance Factor, Tree Height, Applications

Answer Key: At the end

☐ Basics and Properties

1. What is an AVL Tree?
 - A. A binary tree with sorted values
 - B. A balanced binary search tree
 - C. A tree with all leaves at same level
 - D. A heap-based tree

☐

2. What does AVL stand for?
- A. Algorithm Visual List
 - B. Automated Variable List
 - C. Adelson-Velsky and Landis
 - D. Artificial Variable List
- ☐
3. Balance factor of a node in AVL tree is:
- A. Right height – left height
 - B. Left height – right height
 - C. Left subtree nodes – right subtree nodes
 - D. None of the above
- ☐
4. A node is balanced in AVL Tree if balance factor is:
- A. 0
 - B. -1
 - C. +1
 - D. All of the above
- ☐
5. Maximum allowed height difference between left and right subtree in AVL is:
- A. 2
 - B. 0
 - C. 1
 - D. Unlimited
- ☐
-

☐ Rotations in AVL Tree

6. Which of the following rotation is performed for Left-Left imbalance?
- A. Left Rotation
 - B. Right Rotation
 - C. Left-Right Rotation
 - D. Right-Left Rotation
- ☐
7. Which of these is required for Right-Right imbalance?
- A. Left Rotation
 - B. Right Rotation

C. Double Rotation

D. No rotation

☐

8. What kind of rotation is used for Left-Right imbalance?

A. Right-Left Rotation

B. Left Rotation

C. Right Rotation

D. Left-Right Rotation

☐

9. Left-Right rotation consists of:

A. Single right rotation

B. Left then right rotation

C. Right then left rotation

D. Double left rotation

☐

10. How many types of rotations are there in AVL Trees?

A. 2

B. 3

C. 4

D. 5

☐

☐ Insertion & Deletion

11. After inserting a node in AVL tree, rebalancing is done from:

A. Root

B. Leaf

C. Inserted node to root

D. Inserted node to leaf

☐

12. When deleting a node from AVL, the imbalance may occur:

A. Above the deleted node

B. Below deleted node

C. Anywhere

D. Never

☐

13. Insertion into AVL Tree may require:

- A. No rotation
- B. Single rotation
- C. Double rotation
- D. All of the above

☐

14. AVL Tree insertion time complexity is:

- A. $O(1)$
- B. $O(\log n)$
- C. $O(n)$
- D. $O(n \log n)$

☐

15. Deletion time complexity in AVL Tree is:

- A. $O(n)$
- B. $O(n \log n)$
- C. $O(\log n)$
- D. $O(1)$

☐

☐ Tree Properties and Theory

16. What is the minimum number of nodes in AVL tree of height h ?

- A. h
- B. $h + 1$
- C. $\text{Fibonacci}(h+2) - 1$
- D. $2^h - 1$

☐

17. What is the maximum height of an AVL tree with 7 nodes?

- A. 3
- B. 4
- C. 5
- D. 6

☐

18. Which of these trees is most height-balanced?

- A. Binary Search Tree
- B. Red-Black Tree

C. AVL Tree

D. Heap

☐

19. Which tree is faster in insertions and deletions?

A. Red-Black Tree

B. AVL Tree

C. Heap

D. BST

☐

20. When AVL tree becomes unbalanced after deletion, we do:

A. Left rotation

B. Right rotation

C. Rebalance on ancestor

D. Nothing

☐

☐ CUET PG Style / PYQ Pattern

21. ☐ CUET-style: What is the balance factor of a leaf node in AVL Tree?

A. 0

B. 1

C. -1

D. 2

☐

22. ☐ CUET-style: Which of these trees guarantee $O(\log n)$ search time?

A. Binary Tree

B. AVL Tree

C. Linked List

D. Stack

☐

23. ☐ CUET-style: In AVL Tree, after inserting 10, 20, 30, what rotation occurs?

A. Right

B. Left

C. Left-Right

D. Right-Left

☐

24. ☐ CUET-style: What is the height of AVL tree with 15 nodes (approx)?

- A. $\log(15)$
- B. 15
- C. $\sqrt{15}$
- D. None

☐

25. ☐ CUET-style: What is the balance factor range of AVL Tree node?

- A. $\{-2, 0, 2\}$
- B. $\{-1, 0, 1\}$
- C. $\{-3, 0, 3\}$
- D. $\{0, 1, 2\}$

☐

☐ Applications & Misc

26. AVL Trees are used for:

- A. File systems
- B. Compilers
- C. Databases
- D. All of the above

☐

27. Balanced tree ensures:

- A. Better performance
- B. Slower insert
- C. Duplicate handling
- D. None

☐

28. An AVL tree with n nodes has height:

- A. $O(n)$
- B. $O(\log n)$
- C. $O(n^2)$
- D. $O(1)$

☐

29. In AVL Tree, for faster searching:

- A. Height should be more
- B. Height should be less

- C. Nodes should be sorted
- D. Keys must be even

□

30. AVL tree insertion in worst case involves:

- A. No rotation
- B. Single rotation
- C. At most 1 or 2 rotations
- D. Multiple rotations

□

□ Answer Key

1-B 2-C 3-B 4-D 5-C

6-B 7-A 8-D 9-B 10-C

11-C 12-A 13-D 14-B 15-C

16-C 17-B 18-C 19-A 20-C

21-A 22-B 23-B 24-A 25-B

26-D 27-A 28-B 29-B 30-C

□ Topic: B-Trees

Total Questions: 30

Includes: Definitions, Insertions, Deletions, Order, Applications, and Previous Year Style

Answer Key: At the end

□ Basics & Definitions

1. What is a B-Tree?
 - A. A self-balancing binary search tree
 - B. A self-balancing m-ary search tree
 - C. A heap
 - D. A trie

□

2. Who introduced the B-Tree?
- A. Donald Knuth
 - B. Adelson-Velsky and Landis
 - C. Bayer and McCreight
 - D. Dijkstra
- ☐
3. In a B-Tree of order m , a node can have at most:
- A. m children and m keys
 - B. m children and $m-1$ keys
 - C. $m-1$ children and m keys
 - D. $m/2$ keys
- ☐
4. A B-Tree of order m must have at least how many children for a non-root node?
- A. m
 - B. $\lceil m/2 \rceil$
 - C. $m-1$
 - D. 1
- ☐
5. B-Tree is mainly used in:
- A. Operating Systems
 - B. Disk-based storage and file systems
 - C. Compiler Design
 - D. AI
- ☐
-

☐ Structure & Properties

6. Which of the following is false about B-Trees?
- A. All leaves appear on same level
 - B. Keys in node are sorted
 - C. B-Tree can be binary
 - D. Each node can have unlimited children
- ☐
7. What is the time complexity of search in a B-Tree of order m and height h ?
- A. $O(h)$
 - B. $O(m)$

- C. $O(\log m)$
- D. $O(h \log m)$
- ☐

8. Which level are all leaves found in a B-Tree?

- A. Varies
- B. Root level
- C. Same level
- D. Random
- ☐

9. Minimum number of keys in a B-tree node of order m (non-root):

- A. 1
- B. m
- C. $\lceil m/2 \rceil - 1$
- D. $m/2$
- ☐

10. A node with k keys in B-Tree has how many children?

- A. k
 - B. $k+1$
 - C. $2k$
 - D. $k-1$
 - ☐
-

☐ Operations (Insert/Delete)

11. When inserting a key into a B-Tree, if a node overflows:

- A. Delete all keys
- B. Merge it with root
- C. Split the node
- D. Rotate keys
- ☐

12. Splitting a full node during insertion pushes a key:

- A. To the left child
- B. To the root
- C. To the parent
- D. To the right child
- ☐

13. Deletion in B-Trees may cause:
- A. Node merging
 - B. Borrowing keys
 - C. Underflow
 - D. All of these
- ☐
14. Insertion in a B-Tree always starts at:
- A. Root
 - B. Mid-level
 - C. Leaf
 - D. Leftmost child
- ☐
15. What is a common issue handled in B-Trees during deletion?
- A. Overflow
 - B. Underflow
 - C. Duplication
 - D. Loops
- ☐
-

☐ Applications & Uses

16. B-Trees are commonly used in:
- A. Cache memory
 - B. Main memory structures
 - C. File systems and databases
 - D. Graphics
- ☐
17. Why are B-Trees good for databases?
- A. Fast network speed
 - B. Uses binary operations
 - C. Minimizes disk access
 - D. Avoids page faults
- ☐
18. B-Trees handle large data by:
- A. Using recursion
 - B. Storing data sequentially

- C. Reducing tree height
- D. Creating arrays

☐

19. B-Trees are preferred for:

- A. In-memory search
- B. Dynamic arrays
- C. External storage indexing
- D. Graph traversals

☐

20. Which database uses B-Tree indexing?

- A. Oracle
- B. MySQL
- C. PostgreSQL
- D. All of these

☐

☐ CUET PG Style / PYQ Pattern

21. ☐ CUET-style: Which statement about B-Trees is TRUE?

- A. They always remain binary
- B. All keys are stored at root
- C. They allow multiple keys in nodes
- D. Keys are not sorted

☐

22. ☐ CUET-style: A B-Tree of order 5 can have how many maximum keys per node?

- A. 5
- B. 4
- C. 6
- D. 3

☐

23. ☐ CUET-style: What is the minimum number of keys in a non-root node of a B-tree of order 6?

- A. 3
- B. 5
- C. 2

D. 4

☐

24. ☐ CUET-style: In a B-Tree, internal nodes store:

- A. Only children
- B. Only keys
- C. Keys and child pointers
- D. None

☐

25. ☐ CUET-style: Which tree performs better with disk read/writes?

- A. AVL Tree
- B. Binary Tree
- C. B-Tree
- D. Trie

☐

☐ Advanced Concepts

26. What is the maximum number of keys in a B-Tree of order m and height h ?

- A. $m^h - 1$
- B. m^h
- C. $m \times h$
- D. $\log m$

☐

27. If all leaf nodes are at level h in a B-Tree, it means the tree is:

- A. Binary
- B. Balanced
- C. Skewed
- D. Trie

☐

28. B-Trees are not suitable for:

- A. RAM-based operations
- B. Disk-based storage
- C. Indexing
- D. Sorting

☐

29. What makes B-Tree better than BST for large-scale data?

- A. Less space
- B. Faster recursion
- C. Lower height
- D. More height

☐

30. Which one is NOT a variant of B-Tree?

- A. B+ Tree
- B. B* Tree
- C. B# Tree
- D. B- Tree

☐

☐ Answer Key

1-B 2-C 3-B 4-B 5-B

6-D 7-A 8-C 9-C 10-B

11-C 12-C 13-D 14-C 15-B

16-C 17-C 18-C 19-C 20-D

21-C 22-B 23-C 24-C 25-C

26-A 27-B 28-A 29-C 30-D

☐ Topic: B+ Tree

No. of Questions: 30

Includes: Structure, Operations, Differences with B-Trees, Use Cases, and PYQ-style

Answer Key: Provided at the end

☐ Basics & Definitions

1. What is a B+ Tree?

- A. A balanced binary tree
- B. A trie-based tree
- C. A B-Tree where all records are stored at leaf level

D. A max-heap

☐

2. In B+ Trees, where are all actual data records stored?

A. Internal nodes

B. Root node

C. Leaf nodes

D. Randomly across the tree

☐

3. B+ Trees are most useful in:

A. Sorting arrays

B. Operating system scheduling

C. Disk-based databases

D. Stack memory management

☐

4. Which nodes in B+ Trees contain only keys and no actual data?

A. Leaf nodes

B. Root node

C. Internal nodes

D. All nodes

☐

5. What is the order of a B+ Tree?

A. Number of keys in a node

B. Number of children a node can have

C. Number of leaves

D. Tree height

☐

☐ Structure & Properties

6. In a B+ Tree of order m , the number of keys in an internal node is at most:

A. m

B. $m-1$

C. $m+1$

D. $\lceil m/2 \rceil$

☐

7. How many children can a node in a B+ Tree of order m have at most?
- A. $m+1$
 - B. m
 - C. $m-1$
 - D. $m/2$
- ☐
8. What is true about leaf nodes in B+ Trees?
- A. They point to internal nodes
 - B. They are unordered
 - C. They are linked using pointers
 - D. They are deleted regularly
- ☐
9. The B+ Tree is an improvement over B-Tree in:
- A. CPU usage
 - B. Random memory usage
 - C. Range queries
 - D. Stack operations
- ☐
10. Which statement is **false** about B+ Trees?
- A. Leaf nodes contain actual data
 - B. All keys in leaf nodes are sorted
 - C. Internal nodes contain pointers and data
 - D. Leaf nodes are linked
- ☐
-

☐ Operations (Insertion, Search)

11. Search operations in B+ Trees end at:
- A. Root node
 - B. Internal node
 - C. Leaf node
 - D. Random node
- ☐
12. What happens when a leaf node in B+ Tree overflows on insertion?
- A. Deleted
 - B. Rotated

- C. Split
- D. Ignored
- ☐

13. On splitting a full internal node in B+ Tree:

- A. Middle key goes up
- B. All keys move up
- C. Child pointers are ignored
- D. Entire node moves up
- ☐

14. In B+ Tree, which node is updated during deletion?

- A. Only root
- B. Only leaf
- C. Only parent
- D. All affected nodes
- ☐

15. The average time complexity of search in a B+ Tree is:

- A. $O(1)$
 - B. $O(n)$
 - C. $O(\log n)$
 - D. $O(n \log n)$
 - ☐
-

☐ Differences from B-Tree

16. What is a major difference between B+ Tree and B-Tree?

- A. B-Tree is faster
- B. Only B+ Tree stores data in internal nodes
- C. B+ Tree stores data only in leaf nodes
- D. B-Tree is used in disk storage
- ☐

17. Which is better for range queries?

- A. AVL Tree
- B. B-Tree
- C. B+ Tree
- D. Heap
- ☐

18. In B+ Tree, internal nodes:
- A. Contain only data
 - B. Contain pointers and keys
 - C. Contain no keys
 - D. Are not used
- ☐
19. Leaf nodes in B+ Tree are connected in:
- A. Random order
 - B. Left subtree only
 - C. Doubly linked list
 - D. Singly linked list
- ☐
20. Which one provides **better disk-read performance**?
- A. Binary Tree
 - B. AVL Tree
 - C. B+ Tree
 - D. Red-Black Tree
- ☐
-

☐ CUET PG PYQ Style

21. ☐ CUET-style: Which of the following is true for B+ Trees?
- A. Data is stored in both internal and leaf nodes
 - B. All keys are stored only in root
 - C. Internal nodes store only keys
 - D. Leaf nodes store only pointers
- ☐
22. ☐ CUET-style: Which data structure provides fast lookup and range search in DBMS?
- A. Hash Table
 - B. Stack
 - C. B+ Tree
 - D. Heap
- ☐
23. ☐ CUET-style: What is the order of a B+ Tree if max children per node is 5?
- A. 5
 - B. 4

C. 3

D. 6

☐

24. ☐ CUET-style: In B+ Tree, how are the keys in leaf nodes arranged?

A. Randomly

B. In linked list but unsorted

C. Sequentially linked and sorted

D. Tree format

☐

25. ☐ CUET-style: B+ Trees are balanced because:

A. They use min-heaps

B. Height is variable

C. All leaves are at the same level

D. Root has multiple levels

☐

☐ Applications and Advanced

26. B+ Trees are mainly used in:

A. RAM-based systems

B. Compiler design

C. Databases and file systems

D. CPU scheduling

☐

27. Leaf node splitting in B+ Tree involves:

A. Shifting keys to the left

B. Ignoring the split

C. Promoting middle key

D. Deleting all

☐

28. Leaf nodes in B+ Tree are:

A. Redundant

B. Organized in tree format

C. Linked for range queries

D. Used only in binary trees

☐

29. Which of these is a benefit of B+ Tree over B-Tree?

- A. Less disk access
- B. Higher tree height
- C. More unbalanced
- D. Random access

☐

30. How is sequential access optimized in B+ Tree?

- A. Binary search
- B. DFS
- C. Linking leaf nodes
- D. Using graph pointers

☐

☐ Answer Key

1-C 2-C 3-C 4-C 5-B

6-B 7-B 8-C 9-C 10-C

11-C 12-C 13-A 14-D 15-C

16-C 17-C 18-B 19-D 20-C

21-C 22-C 23-A 24-C 25-C

26-C 27-C 28-C 29-A 30-C

☐ Topic: B* Tree

No. of MCQs: 30

Includes: Concepts, properties, operations, differences from B and B+ trees, and PYQ-style

Answer Key: Provided at the end

☐ Basics & Properties

1. What is a B* Tree?

- A. A type of AVL Tree
- B. A variation of B+ Tree
- C. A refined version of B-Tree with better space utilization

D. A threaded binary tree

☐

2. In B* Trees, redistribution of keys happens:

A. After every deletion

B. After every search

C. Before splitting a node

D. Never

☐

3. In comparison to B-Trees, B* Trees offer:

A. Less space usage

B. Better memory utilization and fewer splits

C. More disk access

D. Greater imbalance

☐

4. What's the key difference between B* and B-Trees?

A. B* Tree is not balanced

B. B* redistributes keys before splitting

C. B* Tree supports heap allocation

D. B-Tree has higher order

☐

5. Which of the following is **true** in B* Trees?

A. Each node is always full

B. Nodes split into two equal parts always

C. Nodes try redistribution before splitting

D. Only leaves can split

☐

☐ Node Splitting & Redistribution

6. B* Trees attempt to split:

A. Every time a node is full

B. After redistributing with sibling node

C. When two nodes are full

D. Never

☐

7. How is splitting done in B* Trees?
- A. One node becomes two
 - B. One node becomes three
 - C. One node becomes one
 - D. Parent node is deleted
- ☐
8. B* Tree improves node utilization to:
- A. 50%
 - B. 66%
 - C. 100%
 - D. 40%
- ☐
9. When a node overflows in B* Tree and sibling is not full:
- A. Perform merge
 - B. Perform redistribution
 - C. Split anyway
 - D. Delete the node
- ☐
10. Redistribution in B* Tree improves:
- A. Tree height
 - B. Balance only
 - C. Space efficiency
 - D. None
- ☐
-

☐ Structure & Order

11. What is the **order** of a B* Tree?
- A. Maximum number of leaves
 - B. Maximum number of keys per node
 - C. Maximum children per node
 - D. Tree height
- ☐
12. In B* Tree of order m , max keys in an internal node is:
- A. $m-1$
 - B. m

C. $m+1$

D. $m/2$

☐

13. In B* Tree of order m , max children in a node is:

A. m

B. $m-1$

C. $m+1$

D. $m/2$

☐

14. When splitting occurs in B* Tree:

A. It increases height always

B. Redistributes among 3 nodes

C. Redistributes among 2 nodes

D. Redistributes among leaves only

☐

15. Which of the following is used more in disk-based file systems?

A. AVL

B. B-Tree

C. B* Tree

D. Binary Tree

☐

☐ Comparison with Other Trees

16. Which is more space-efficient:

A. B-Tree

B. B+ Tree

C. B* Tree

D. Binary Tree

☐

17. B* Trees are more efficient than B-Trees because:

A. They require fewer node splits

B. They are faster in search

C. They are linked

D. They are binary

☐

18. B* Tree differs from B+ Tree by:

- A. Not storing data in leaves
- B. Redistribution before split
- C. Sorting in internal nodes
- D. Using hashing

☐

19. B* Tree insertion is more complex than B-Tree because:

- A. B* Trees are unbalanced
- B. Requires redistribution logic
- C. Involves binary search
- D. Only balanced on deletion

☐

20. B* Tree is a type of:

- A. Binary Search Tree
- B. Self-balancing tree
- C. Static tree
- D. Array-based tree

☐

☐ Operations (Search, Insert, Delete)

21. Searching in a B* Tree:

- A. Ends at root
- B. Is slower than B-Tree
- C. Is same as B-Tree
- D. Is done using DFS

☐

22. Insertions in B* Tree may require:

- A. Sibling check for redistribution
- B. Always splitting
- C. Rebalancing root only
- D. Preorder traversal

☐

23. Which node type may be split in B* Tree?

- A. Only root
- B. Only internal

C. Any full node

D. None

☐

24. Leaf node split in B* Tree is handled:

A. By deleting a key

B. Using redistribution or split

C. By rotating

D. Through graph traversal

☐

25. Which of the following happens **first** during insertion in B* Tree?

A. Split the node

B. Delete from root

C. Redistribute with sibling

D. Merge

☐

☐ CUET PG PYQ Style & Application

26. ☐ CUET-style: Which of these has **better utilization of disk space**?

A. AVL Tree

B. B+ Tree

C. B Tree

D. B* Tree

☐

27. ☐ CUET-style: Redistribution of keys before splitting is a property of:

A. B-Tree

B. Red-Black Tree

C. B* Tree

D. AVL Tree

☐

28. ☐ CUET-style: In a B* Tree, if redistribution is not possible:

A. Insert fails

B. Search is repeated

C. Node is split

D. Root is deleted

☐

29. □ CUET-style: B* Trees aim to keep nodes:

- A. Half full
- B. Two-thirds full
- C. Randomly filled
- D. Fully empty

□

30. □ CUET-style: Which tree performs **redistribution** before split to reduce tree height growth?

- A. B-Tree
- B. AVL Tree
- C. B+ Tree
- D. B* Tree

□

□ Answer Key

1-C 2-C 3-B 4-B 5-C

6-B 7-B 8-B 9-B 10-C

11-C 12-A 13-A 14-B 15-C

16-C 17-A 18-B 19-B 20-B

21-C 22-A 23-C 24-B 25-C

26-D 27-C 28-C 29-B 30-D

□ Topic: Data Structures for Sets

No. of MCQs: 30

Includes: Arrays, Linked Lists, Hash Tables, Bit Vectors, Disjoint Sets (Union-Find)

Answer Key: At the end

□ Basic Concepts

1. Which of the following is **not** a typical operation on a set?

- A. Union
- B. Find

C. Insertion

D. Rotation

☐

2. Which data structure is commonly used for **dynamic sets**?

A. Stack

B. Queue

C. Hash Table

D. Tree

☐

3. Which of the following is **best** for implementing a static set?

A. Linked list

B. Array

C. Tree

D. Hashing

☐

4. **Bit vector** representation of sets is most efficient when:

A. Elements are large integers

B. Set has many duplicates

C. Universe of elements is small

D. Elements are strings

☐

5. What is the main drawback of **bit vector** representation?

A. Complex logic

B. Poor performance

C. Space inefficiency for large universes

D. Slow access time

☐

☐ Array & Linked List Implementation

6. What is the time complexity of **searching** in an unsorted array used for a set?

A. $O(1)$

B. $O(\log n)$

C. $O(n)$

D. $O(n \log n)$

☐

7. Inserting an element in a set represented as a **sorted array** takes:
- A. $O(1)$
 - B. $O(\log n)$
 - C. $O(n)$
 - D. $O(n^2)$
- ☐
8. Which operation is easier in linked list set representation?
- A. Sorting
 - B. Binary search
 - C. Union
 - D. In-place reversal
- ☐
9. If sets are implemented using **linked lists**, the complexity of the **union** operation is:
- A. $O(n)$
 - B. $O(n \log n)$
 - C. $O(n^2)$
 - D. $O(1)$
- ☐
10. What is the major benefit of using **linked lists** over arrays for sets?
- A. Faster traversal
 - B. Fixed size
 - C. Dynamic size
 - D. Better sorting
- ☐
-

☐ Disjoint Set (Union-Find)

11. Disjoint Set Data Structure supports:
- A. DFS & BFS
 - B. Find and Union
 - C. Sort and Merge
 - D. Insert and Delete
- ☐
12. Union-Find data structure is mainly used in:
- A. Dynamic programming
 - B. Graph algorithms

C. Recursion

D. Sorting

☐

13. Union by rank and path compression are used to:

A. Increase memory

B. Slow down operations

C. Optimize find operation

D. Avoid recursion

☐

14. In Disjoint Sets, **Find** operation returns:

A. Minimum element

B. Maximum element

C. Representative of the set

D. Length of set

☐

15. Time complexity of Find operation with path compression is:

A. $O(n)$

B. $O(\log n)$

C. $O(\alpha(n))$

D. $O(1)$

☐

(Where $\alpha(n)$ is the inverse Ackermann function)

☐ Hash Table in Sets

16. Which data structure gives **average $O(1)$** time for insert, delete, and search?

A. Tree

B. Linked List

C. Hash Table

D. Graph

☐

17. Hashing is **not** suitable for:

A. Checking membership

B. Sorting

C. Searching

D. Insertion

☐

18. Which problem may arise in hashing?

A. Underflow

B. Collision

C. Overflow

D. Segmentation fault

☐

19. Separate chaining in hashing uses:

A. Arrays

B. Linked Lists

C. Queues

D. Graphs

☐

20. Load factor in hashing is defined as:

A. n/m

B. m/n

C. $n + m$

D. $m \times n$

☐

(Where n = number of elements, m = number of buckets)

☐ CUET PYQ-style & Application

21. ☐ CUET-style: Which structure allows efficient **union** and **find**?

A. Hash Map

B. AVL Tree

C. Disjoint Set

D. Array

☐

22. ☐ CUET-style: Bit vector implementation of set assumes:

A. Elements are floating point

B. Infinite universe

C. Fixed finite universe

D. Non-integer elements

☐

23. ☐ CUET-style: Union by rank aims to:

- A. Increase search time
- B. Decrease depth of tree
- C. Build larger trees
- D. None

☐

24. ☐ CUET-style: Path compression is applied during:

- A. Insert
- B. Find
- C. Delete
- D. Merge

☐

25. ☐ CUET-style: What is the worst-case complexity of Find without path compression?

- A. $O(1)$
- B. $O(n)$
- C. $O(\log n)$
- D. $O(n \log n)$

☐

26. ☐ CUET-style: Hashing performs poorly when:

- A. Load factor is small
- B. Collisions are handled
- C. Many collisions occur
- D. None

☐

27. ☐ CUET-style: For small fixed-size set with known universe, the best implementation is:

- A. Binary Tree
- B. Linked List
- C. Bit Vector
- D. Hash Table

☐

28. ☐ CUET-style: Find(x) in disjoint sets returns:

- A. Parent of x
- B. Root of tree containing x
- C. Next sibling
- D. Nothing

☐

29. ☐ CUET-style: In union-find, a tree with rank 2 means:
- A. Tree height is 2
 - B. Tree has 2 nodes
 - C. Tree has 3 leaves
 - D. Unknown
- ☐
30. ☐ CUET-style: A collision in hash table can be resolved using:
- A. Recursion
 - B. Stacks
 - C. Linear probing
 - D. Trees
- ☐
-

☐ Answer Key

1-D 2-C 3-B 4-C 5-C

6-C 7-C 8-C 9-A 10-C

11-B 12-B 13-C 14-C 15-C

16-C 17-B 18-B 19-B 20-A

21-C 22-C 23-B 24-B 25-B

26-C 27-C 28-B 29-A 30-C

Here are **30 MCQs on Graphs** — ideal for **CUET PG MCA** preparation — including theory-based, code-based, and CUET-style previous year question types.

☐ Topic: Graphs

Total MCQs: 30

Coverage: Representation, Traversal (DFS/BFS), Types, Applications, Algorithms (Dijkstra, Kruskal, Prim, etc.)

Answer Key: Provided at the end

Includes PYQ-style questions

□ Basics and Representation

1. A graph is a collection of:
 - A. Nodes only
 - B. Edges only
 - C. Nodes and edges
 - D. Trees□
2. In an undirected graph with n nodes, the **maximum** number of edges is:
 - A. n
 - B. $n(n-1)/2$
 - C. n^2
 - D. $n(n+1)/2$□
3. A complete graph with n vertices has how many edges?
 - A. n
 - B. $n(n-1)/2$
 - C. $n(n+1)/2$
 - D. $2n$□
4. An adjacency matrix for a **graph with n vertices** requires:
 - A. n
 - B. n^2
 - C. $2n$
 - D. $\log(n)$□
5. Which representation is most **space-efficient** for sparse graphs?
 - A. Adjacency Matrix
 - B. Adjacency List
 - C. Incidence Matrix
 - D. 2D Array□
6. The time complexity to check if there is an edge between two nodes in an **adjacency matrix** is:
 - A. $O(n)$
 - B. $O(\log n)$
 - C. $O(1)$

D. $O(n^2)$

☐

7. A graph with **no cycles and connected** is a:

A. DAG

B. Tree

C. Complete Graph

D. Subgraph

☐

8. Which of the following graphs must be **acyclic**?

A. Directed Graph

B. Undirected Graph

C. Tree

D. Bipartite

☐

☐ Graph Traversals (BFS/DFS)

9. Breadth-first search uses which data structure?

A. Stack

B. Queue

C. Array

D. HashMap

☐

10. Depth-first search uses which data structure?

A. Queue

B. Stack

C. Linked List

D. Tree

☐

11. Time complexity of BFS and DFS is:

A. $O(n^2)$

B. $O(n \log n)$

C. $O(V + E)$

D. $O(V^2)$

☐

12. Which of the following algorithms is **not** based on DFS?

- A. Topological Sort
- B. Finding Strongly Connected Components
- C. Cycle Detection
- D. Dijkstra's Algorithm

☐

13. DFS traversal is best suited for:

- A. Shortest path
- B. All-pairs path
- C. Topological sort
- D. Level order traversal

☐

☐ Types of Graphs

14. A **directed graph** where each edge has weight is called:

- A. Directed Acyclic Graph
- B. Weighted Graph
- C. Tree
- D. Bipartite

☐

15. A graph is **cyclic** if:

- A. All nodes are connected
- B. Every node has 2 neighbors
- C. It contains a closed path
- D. It contains only trees

☐

16. A **bipartite graph** must have:

- A. Cycles
- B. No cycles
- C. Two disjoint vertex sets
- D. Equal edges

☐

17. A graph is connected if:

- A. There are no isolated nodes
- B. There are no cycles

- C. Every node has at least 2 edges
 - D. There is a path between every pair of vertices
- ☐

☐ Algorithms on Graphs

18. Dijkstra's algorithm is used to:
- A. Detect cycles
 - B. Find MST
 - C. Find shortest paths from one source
 - D. Sort vertices
- ☐
19. Dijkstra's algorithm fails with:
- A. Positive weights
 - B. Zero weights
 - C. Negative weights
 - D. Dense graphs
- ☐
20. What data structure is used in Dijkstra's algorithm?
- A. Queue
 - B. Stack
 - C. Priority Queue (Min-Heap)
 - D. Graph
- ☐
21. Which algorithm guarantees a **Minimum Spanning Tree**?
- A. Dijkstra
 - B. BFS
 - C. DFS
 - D. Kruskal
- ☐
22. Kruskal's algorithm uses:
- A. Priority Queue
 - B. Union-Find
 - C. Stack
 - D. Matrix
- ☐

23. Which algorithm does **not** work on directed graphs?

- A. Prim's
- B. BFS
- C. DFS
- D. Topological sort

☐

24. What is the output of **Topological Sorting**?

- A. MST
- B. DFS tree
- C. Linear ordering of vertices
- D. Shortest path

☐

25. Bellman-Ford can handle:

- A. Only positive weights
- B. Only unweighted graphs
- C. Negative weights
- D. Cycles only

☐

☐ CUET PG PYQ-style Questions

26. ☐ CUET PYQ-style: A graph with 10 vertices and 20 edges is represented using an adjacency matrix. What is the space required?

- A. 10
- B. 20
- C. 100
- D. 200

☐

27. ☐ CUET PYQ-style: Which graph algorithm is used in detecting **cycle** in a directed graph?

- A. Prim's
- B. Dijkstra's
- C. DFS
- D. BFS

☐

28. ☐ CUET PYQ-style: A graph where edges have direction is called:

- A. Directed

B. Undirected

C. Tree

D. Bipartite

□

29. □ CUET PYQ-style: In DFS, a node is visited:

A. Level-wise

B. Breadth-wise

C. Depth-wise

D. Randomly

□

30. □ CUET PYQ-style: The best algorithm for **single-source shortest path** in graphs with **negative weights** is:

A. Prim's

B. Dijkstra

C. Kruskal

D. Bellman-Ford

□

□ Answer Key

1-C 2-B 3-B 4-B 5-B

6-C 7-B 8-C 9-B 10-B

11-C 12-D 13-C 14-B 15-C

16-C 17-D 18-C 19-C 20-C

21-D 22-B 23-D 24-C 25-C

26-C 27-C 28-A 29-C 30-D

□ Topic: Sorting and Searching Algorithms

Total MCQs: 30

Includes: Basics, Bubble/Quick/Merge/Heap Sort, Binary/Linear Search, Time complexities

PYQs: Included

Answers: At end

☐ Basics & Time Complexity

1. Which sorting algorithm has the **best average case time complexity**?
 - A. Bubble Sort
 - B. Selection Sort
 - C. Merge Sort
 - D. Insertion Sort☐
2. Time complexity of **Merge Sort** in worst case is:
 - A. $O(n^2)$
 - B. $O(n \log n)$
 - C. $O(n)$
 - D. $O(\log n)$☐
3. Which sorting algorithm is **in-place and stable**?
 - A. Merge Sort
 - B. Bubble Sort
 - C. Quick Sort
 - D. Heap Sort☐
4. Which sorting algorithm is **not** comparison-based?
 - A. Merge Sort
 - B. Counting Sort
 - C. Quick Sort
 - D. Heap Sort☐
5. Which of the following **does not maintain stability**?
 - A. Insertion Sort
 - B. Merge Sort
 - C. Quick Sort
 - D. Bubble Sort☐
6. Which algorithm divides input into **smaller subproblems**?
 - A. Merge Sort
 - B. Bubble Sort
 - C. Selection Sort

D. Insertion Sort

☐

7. Time complexity of Bubble Sort in best case:

A. $O(n^2)$

B. $O(n \log n)$

C. $O(n)$

D. $O(1)$

☐

8. Which sorting algorithm works by **selecting the minimum element** repeatedly?

A. Quick Sort

B. Insertion Sort

C. Selection Sort

D. Merge Sort

☐

9. Which sorting algorithm uses a **pivot**?

A. Heap Sort

B. Merge Sort

C. Quick Sort

D. Bubble Sort

☐

10. In Heap Sort, which data structure is used?

A. Stack

B. Queue

C. Heap

D. Tree

☐

☐ Binary and Linear Search

11. Time complexity of **Linear Search** in worst case:

A. $O(n \log n)$

B. $O(1)$

C. $O(n)$

D. $O(\log n)$

☐

12. Binary Search requires:

- A. Sorted array
- B. Unsorted array
- C. Graph
- D. Queue

☐

13. What is the worst-case time complexity of Binary Search?

- A. $O(\log n)$
- B. $O(n \log n)$
- C. $O(n)$
- D. $O(1)$

☐

14. In Binary Search, if we halve the array each time, what is the depth of recursion?

- A. $\log_2 n$
- B. n
- C. $\sqrt{n}$
- D. n^2

☐

15. Binary Search cannot be applied on:

- A. Sorted Arrays
- B. Balanced Trees
- C. Linked List
- D. Ordered Set

☐

☐ Quick Sort vs Merge Sort

16. Quick Sort is faster than Merge Sort because:

- A. Uses heap
- B. Divide and conquer
- C. Better cache performance
- D. Stable

☐

17. Worst-case time complexity of Quick Sort:

- A. $O(n^2)$
- B. $O(n \log n)$

C. $O(n)$

D. $O(1)$

☐

18. Quick Sort is **not stable** because:

A. Extra memory is required

B. Swapping is non-deterministic

C. Recursion is used

D. It uses pivot

☐

19. Merge Sort uses:

A. Stack

B. Heap

C. Queue

D. Recursion

☐

20. Which sorting algorithm is best for linked lists?

A. Merge Sort

B. Quick Sort

C. Heap Sort

D. Bubble Sort

☐

☐ Heap and Counting Sort

21. In Heap Sort, after building the heap, the time taken to sort is:

A. $O(n \log n)$

B. $O(n^2)$

C. $O(n)$

D. $O(1)$

☐

22. Which sorting algorithm is **non-comparison based**?

A. Quick Sort

B. Merge Sort

C. Heap Sort

D. Radix Sort

☐

23. Counting Sort is **not suitable** for:

- A. Floating point numbers
- B. Integers
- C. Small range inputs
- D. Stable sort requirement

☐

24. Counting Sort has time complexity:

- A. $O(n^2)$
- B. $O(n \log n)$
- C. $O(n + k)$
- D. $O(\log n)$

☐

25. Which sorting algorithm is best suited for small data sets and nearly sorted data?

- A. Merge Sort
- B. Insertion Sort
- C. Heap Sort
- D. Quick Sort

☐

☐ CUET PG PYQ-style Questions

26. ☐CUET PYQ-style: Binary search algorithm can be used only on:

- A. Sorted data
- B. Unsorted data
- C. Trees only
- D. Strings only

☐

27. ☐CUET PYQ-style: Which sorting algorithm gives best performance on average case?

- A. Selection
- B. Quick
- C. Merge
- D. Bubble

☐

28. ☐CUET PYQ-style: What is the time complexity of linear search in worst case?

- A. $O(\log n)$
- B. $O(n)$

C. $O(n \log n)$

D. $O(1)$

☐

29. ☐ CUET PYQ-style: Which of the following sorting algorithms is **not stable**?

A. Merge

B. Insertion

C. Quick

D. Bubble

☐

30. ☐ CUET PYQ-style: Heap sort is based on which data structure?

A. Stack

B. Queue

C. Binary Heap

D. Tree

☐

☐ Answer Key

1-C 2-B 3-B 4-B 5-C

6-A 7-C 8-C 9-C 10-C

11-C 12-A 13-A 14-A 15-C

16-C 17-A 18-B 19-D 20-A

21-A 22-D 23-A 24-C 25-B

26-A 27-B 28-B 29-C 30-C

☐ Topic: Hashing

Total MCQs: 30

Includes: Hash functions, collision resolution, chaining, open addressing, load factor

CUET PYQ-based: Included

Answer key: At the end

☐ Hashing Basics

1. Hashing is used for:
 - A. Sorting data
 - B. Compressing data
 - C. Fast data retrieval
 - D. Encryption☐
 2. A good hash function should:
 - A. Minimize collisions
 - B. Be complex
 - C. Use recursion
 - D. Be hardware dependent☐
 3. The time complexity of searching in a hash table (average case) is:
 - A. $O(n \log n)$
 - B. $O(\log n)$
 - C. $O(1)$
 - D. $O(n)$☐
 4. The time complexity of searching in a hash table (worst case) is:
 - A. $O(1)$
 - B. $O(\log n)$
 - C. $O(n)$
 - D. $O(n^2)$☐
 5. Which is not a collision resolution strategy?
 - A. Chaining
 - B. Linear probing
 - C. Binary Search
 - D. Quadratic probing☐
-

☐ Collision Resolution

6. In chaining method, the data is stored in:
 - A. Arrays

- B. Stacks
- C. Linked Lists
- D. Queues

☐

7. In open addressing, all elements are stored in:

- A. Linked list
- B. Hash table itself
- C. External array
- D. BST

☐

8. What is the probe sequence for **linear probing**?

- A. $h(k)$, $h(k)+1$, $h(k)+2$,...
- B. $h(k)$, $h(k)+1^2$, $h(k)+2^2$,...
- C. $h(k)$, $h(k)*2$, $h(k)*3$,...
- D. $h(k)$, $h(k)+\log(k)$,...

☐

9. In **quadratic probing**, the probing sequence is:

- A. Linear
- B. Exponential
- C. Quadratic
- D. Logarithmic

☐

10. What problem occurs in linear probing when many elements are clustered?

- A. Rehashing
- B. Primary clustering
- C. Infinite loop
- D. Deadlock

☐

☐ Hash Functions

11. The division method for hashing computes index as:

- A. $\text{key \% table_size}$
- B. key / table_size
- C. table_size / key

D. key + table_size

☐

12. If hash table size is a power of 2, it may lead to:

A. Speed up

B. More collisions

C. Less collisions

D. Uniform distribution

☐

13. A hash function maps:

A. Keys to values

B. Keys to indices

C. Values to keys

D. Keys to tables

☐

14. Which hashing technique is best when the data is sequential?

A. Linear probing

B. Quadratic probing

C. Chaining

D. Double hashing

☐

15. Which hashing method avoids clustering best?

A. Linear Probing

B. Quadratic Probing

C. Chaining

D. Double Hashing

☐

☐ Load Factor, Rehashing

16. Load factor (α) is defined as:

A. Number of elements / table size

B. Table size / number of elements

C. Number of collisions / total insertions

D. Total insertions / collisions

☐

17. What happens if load factor increases beyond threshold?

- A. Deletion
- B. Sorting
- C. Rehashing
- D. Clustering

☐

18. Rehashing involves:

- A. Compressing table
- B. Resizing and re-inserting elements
- C. Removing duplicates
- D. Changing hash values

☐

19. Which scenario requires rehashing?

- A. Low collisions
- B. Load factor < 0.5
- C. Too many collisions
- D. Sorted keys

☐

20. When is hash table performance best?

- A. Load factor > 1
- B. Load factor $= 1$
- C. Load factor < 1
- D. Load factor $= 0$

☐

☐ CUET PG PYQ-Style & Practice

21. ☐ CUET PYQ-style: Hashing technique is used primarily for:

- A. Searching
- B. Sorting
- C. Indexing
- D. Traversing

☐

22. In chaining, the average search time depends on:

- A. Table size
- B. Load factor

C. Number of probes

D. Key value

☐

23. Double hashing is used to:

A. Avoid primary clustering

B. Avoid secondary clustering

C. Avoid both

D. None

☐

24. A collision in hashing means:

A. Two elements hashed to same index

B. Hash table is full

C. Array overflow

D. Function not working

☐

25. Which of these can cause poor performance in a hash table?

A. Low load factor

B. Many collisions

C. Sorted keys

D. Short keys

☐

☐ Miscellaneous and Applications

26. What is the hash value of key = 42 if table size = 10 (division method)?

A. 0

B. 2

C. 4

D. 6

☐

27. A perfect hash function has:

A. No collisions

B. Minimum collisions

C. Unique key

D. Equal-sized buckets

☐

28. What happens when a key is not found in open addressing?

- A. Error
- B. Continue probing until empty cell
- C. Loop terminates
- D. Start rehashing

☐

29. Which of the following is true?

- A. Hashing uses binary search
- B. Hashing gives $O(1)$ search time on average
- C. Hashing requires sorted data
- D. Hashing is only for numbers

☐

30. Which is NOT a valid hashing method?

- A. Mid-square
- B. Folding
- C. Chaining
- D. Merging

☐

☐ **Answer Key**

1-C 2-A 3-C 4-C 5-C

6-C 7-B 8-A 9-C 10-B

11-A 12-B 13-B 14-D 15-D

16-A 17-C 18-B 19-C 20-C

21-A 22-B 23-C 24-A 25-B

26-B 27-A 28-B 29-B 30-D

☐ **Topic: Functions**

Total Questions: 30

Covers: Function types, recursion, scope, storage classes, calling mechanisms, parameters

CUET PYQ-style questions: Included

Answer key: At the end

☐ Basics of Functions

1. A function is:
 - A. A collection of variables
 - B. A group of statements performing a specific task
 - C. A looping construct
 - D. A reserved keyword☐
 2. What is the return type of a function that does not return a value?
 - A. void
 - B. null
 - C. int
 - D. float☐
 3. The default return type of a function in C (if not specified) is:
 - A. void
 - B. int
 - C. float
 - D. undefined☐
 4. A function can return:
 - A. Only one value
 - B. Multiple values
 - C. Only strings
 - D. No value☐
 5. A function prototype tells the compiler about:
 - A. Function definition
 - B. Number and types of parameters
 - C. Return statement
 - D. Main program☐
-

☐ Function Types and Usage

6. A user-defined function is:
- A. Built-in function
 - B. Defined by programmer
 - C. Always recursive
 - D. Always inline
- ☐
7. Which of the following is a valid function declaration in C?
- A. `function int();`
 - B. `int function();`
 - C. `int = function();`
 - D. `func int();`
- ☐
8. A recursive function is one that:
- A. Calls other functions
 - B. Calls itself
 - C. Executes forever
 - D. Cannot return values
- ☐
9. What will happen if a recursive function lacks a base condition?
- A. Infinite recursion
 - B. Compilation error
 - C. Returns 0
 - D. Executes once
- ☐
10. The value returned by a function is:
- A. Always a float
 - B. Stored in main
 - C. Passed using return statement
 - D. Printed automatically
- ☐
-

☐ Parameters and Arguments

11. Which function call method copies actual parameters to formal parameters?
- A. Call by reference
 - B. Call by value

- C. Call by address
- D. Call by assignment

☐

12. Call by reference allows:

- A. Only reading the value
- B. Modifying original value
- C. Declaring variables
- D. Printing address only

☐

13. In C, functions are:

- A. Always called by value
- B. Always called by reference
- C. Only void
- D. Not allowed

☐

14. Which of the following allows returning multiple values from a function?

- A. Call by reference
- B. Call by value
- C. Global variables only
- D. Returning an int

☐

15. Which keyword is used to send back value from function?

- A. send
- B. return
- C. back
- D. give

☐

☐ Scope and Storage Classes

16. A local variable is accessible:

- A. Throughout the program
- B. Inside a function only
- C. In header file
- D. In the main() only

☐

17. A global variable is:

- A. Declared inside a function
- B. Accessible everywhere
- C. Used only in loops
- D. Not allowed in C

☐

18. Which storage class stores variables in memory but maintains value between function calls?

- A. auto
- B. register
- C. static
- D. extern

☐

19. Which storage class is used for external linkage between files?

- A. auto
- B. static
- C. extern
- D. global

☐

20. Which storage class suggests storing variable in CPU register?

- A. register
- B. static
- C. extern
- D. volatile

☐

☐ CUET PYQ-Style and Practice

21. ☐CUET PYQ-style: In C, recursion is mainly implemented through:

- A. Loops
- B. Switch
- C. Function calls
- D. Pointers

☐

22. What is the output of the following code?

```
int fun() { static int x=10; x++; return x; }
```

```
int main() { printf("%d", fun()); printf("%d", fun()); }
```

- A. 11 12
- B. 10 11
- C. 12 13
- D. 11 11

☐

23. Which of the following is not a valid function?

- A. int 123fun()
- B. int fun123()
- C. void fun()
- D. int _fun()

☐

24. Inline functions are mainly used to:

- A. Reduce execution time
- B. Reduce memory
- C. Create infinite loops
- D. Improve I/O

☐

25. The stack frame of a function call stores:

- A. Return address
- B. Parameters
- C. Local variables
- D. All of the above

☐

☐ Miscellaneous

26. The number of return values a function can have in C:

- A. 0
- B. 1
- C. 2
- D. Infinite

☐

27. Recursive functions use which data structure internally?

- A. Queue
- B. Stack

C. Heap

D. Array

☐

28. If a function has both local and global variable with same name, which is used inside function?

A. Global

B. Local

C. Both

D. Depends on compiler

☐

29. Inline functions are expanded at:

A. Runtime

B. Compile time

C. Linking

D. Never

☐

30. The function main() in C returns:

A. void

B. int

C. string

D. double

☐

☐ Answer Key

1-B 2-A 3-B 4-A 5-B

6-B 7-B 8-B 9-A 10-C

11-B 12-B 13-A 14-A 15-B

16-B 17-B 18-C 19-C 20-A

21-C 22-A 23-A 24-A 25-D

26-B 27-B 28-B 29-B 30-B

☐ Topic: Recursion

Total MCQs: 30

Includes: Concepts, tracing, time complexity, applications, and CUET PYQ-style

Answer key: At the end

□ Conceptual Basics

1. What is recursion in programming?
 - A. A function that calls another function
 - B. A loop that repeats indefinitely
 - C. A function calling itself
 - D. A global function□
2. What is essential for a recursive function to terminate?
 - A. Function parameters
 - B. Loop
 - C. Base case
 - D. Global variable□
3. What is the main advantage of recursion?
 - A. Slower execution
 - B. Increased code length
 - C. Compact code and easier problem solving
 - D. Higher memory use□
4. A recursive function without base condition results in:
 - A. Compilation error
 - B. Infinite loop
 - C. Stack overflow
 - D. Both B and C□
5. Which data structure is used to implement recursion internally?
 - A. Queue
 - B. Stack
 - C. Linked list
 - D. Heap□

☐ Code Tracing

6. How many times will this function call itself?

```
void fun(int n) {  
    if(n == 0) return;  
    fun(n-1);  
}
```

fun(5);

A. 4

B. 5

C. 6

D. Infinite

☐

7. What is the output of the following recursive code?

```
int fun(int n) {  
    if(n == 0) return 0;  
    return n + fun(n-1);  
}
```

fun(3);

A. 3

B. 6

C. 9

D. 0

☐

8. What will be the output of this code?

```
void print(int n) {  
    if(n == 0) return;  
    print(n - 1);  
    printf("%d ", n);  
}
```

}

print(3);

A. 3 2 1

B. 1 2 3

C. 0 1 2

D. 2 1 0

☐

9. What does the following function compute?

```
int fun(int n) {
```

```
    if(n <= 1) return 1;
```

```
    return fun(n-1) + fun(n-2);
```

```
}
```

A. Factorial

B. Fibonacci

C. GCD

D. LCM

☐

10. Which of the following problems is best solved using recursion?

A. Calculating average

B. Searching in array

C. Tree traversal

D. Iteration over list

☐

☐ CUET PG PYQ Style

11. ☐ CUET PYQ-style: Which of the following must be true for recursion to work properly?

A. Function must be void

B. No return value

C. Must have base condition

D. Infinite calls

☐

12. Recursive function requires:

- A. Only loop
- B. Base condition and recursive call
- C. Static memory
- D. goto statements

☐

13. Which of these is tail recursion?

```
int fun(int n) {  
    if(n==0) return 0;  
    return fun(n-1);  
}
```

- A. Yes
- B. No
- C. Only if n is odd
- D. Cannot say

☐

14. What happens if a recursive function keeps calling itself without base case?

- A. Segmentation fault
- B. Infinite loop
- C. Stack overflow
- D. All of these

☐

15. What is the time complexity of Fibonacci(n) using recursion?

- A. $O(n)$
- B. $O(\log n)$
- C. $O(2^n)$
- D. $O(n^2)$

☐

☐ Applications & Theory

16. Which problem is **not suitable** for recursion?

- A. Tower of Hanoi
- B. Binary Tree Traversal

- C. Factorial
- D. Bubble Sort
- ☐

17. What is the major drawback of recursion?

- A. Less accuracy
- B. Slower speed
- C. High memory usage
- D. Syntax error
- ☐

18. Recursion is **not preferred** when:

- A. Solution requires branching
- B. Iterative solution is more efficient
- C. Stack is empty
- D. You use loops
- ☐

19. In recursion, each function call uses:

- A. Same memory block
- B. Separate memory stack frame
- C. Queue
- D. Pointer
- ☐

20. Which of the following uses **direct recursion**?

- A. Function A calls function B
- B. Function A calls itself
- C. Function A calls B, B calls C, C calls A
- D. None
- ☐

☐ Programming Questions

21. What is factorial(3)?

```
int fact(int n) {  
    if(n <= 1) return 1;  
    return n * fact(n-1);  
}
```

}

A. 1

B. 3

C. 6

D. 0

☐

22. Count how many times fun is called:

```
void fun(int n) {
```

```
    if(n > 0) {
```

```
        fun(n-1);
```

```
        fun(n-1);
```

```
    }
```

```
}
```

```
fun(3);
```

A. 7

B. 15

C. 14

D. 13

☐

23. What is the output?

```
int fun(int x) {
```

```
    if(x <= 0) return 0;
```

```
    else return x + fun(x - 1);
```

```
}
```

```
fun(4);
```

A. 6

B. 10

C. 12

D. 8

☐

24. The recursive depth for computing factorial(n) is:

- A. n
- B. n-1
- C. log(n)
- D. 2^n

☐

25. What happens to stack frame when recursion ends?

- A. Cleared one by one
- B. All remain
- C. Loop continues
- D. Stored in heap

☐

☐ Miscellaneous

26. Which is **not** a type of recursion?

- A. Direct
- B. Indirect
- C. Infinite
- D. Ternary

☐

27. Which of these is an example of **indirect recursion**?

- A. A() calls A()
- B. A() calls B(), B() calls A()
- C. A() calls A(), B() calls A()
- D. A() and B() never call

☐

28. Which is more efficient in memory?

- A. Iteration
- B. Recursion
- C. Depends
- D. None

☐

29. When using recursion, which of the following is **essential**?

- A. Stack
- B. Global variable

C. goto

D. loop

□

30. Recursion is **generally preferred** for:

A. Linear search

B. Tree traversal

C. Sorting

D. File handling

□

□ Answer Key

1-C 2-C 3-C 4-D 5-B

6-B 7-B 8-B 9-B 10-C

11-C 12-B 13-A 14-D 15-C

16-D 17-C 18-B 19-B 20-B

21-C 22-B 23-B 24-A 25-A

26-D 27-B 28-A 29-A 30-B

□ Topic: Parameter Passing

Total MCQs: 30

Includes: Concepts, programming examples, call by value/reference, CUET PYQ-style

Answer Key: At the end

□ Basics of Parameter Passing

1. What is the purpose of parameter passing in functions?

A. To repeat a function

B. To pass values to a function

C. To store data globally

D. To avoid using functions

□

2. Which of the following parameter passing techniques does **C language** support?

- A. Call by reference only
- B. Call by value only
- C. Call by value and call by reference
- D. Call by name

☐

3. In call by value:

- A. Address of variable is passed
- B. Copy of value is passed
- C. No data is passed
- D. Variable is modified globally

☐

4. In call by reference:

- A. Copy of variable is passed
- B. Reference or address is passed
- C. Only constants can be passed
- D. It is not allowed in C

☐

5. Which one of the following languages uses **call by value** by default?

- A. C++
- B. Java
- C. Python
- D. C

☐

☐ Tracing with Functions

6. What will be the output of this code?

```
void fun(int x) {
```

```
    x = x + 5;
```

```
}
```

```
int main() {
```

```
    int a = 10;
```

```
    fun(a);
```

```
printf("%d", a);  
}
```

- A. 15
 - B. 10
 - C. 5
 - D. Error
- ☐

7. Which of the following passes **actual address** of variables?

- A. Call by value
- B. Call by reference
- C. Pass by copy
- D. Pass by constant

☐

8. What is the effect of call by value on the actual variable?

- A. Modifies it
- B. Deletes it
- C. No effect
- D. Converts to string

☐

9. What is required to implement call by reference in C?

- A. Loop
- B. Pointer
- C. Array
- D. Constant

☐

10. What is the output of this program?

```
void swap(int *a, int *b) {  
  
    int temp = *a;  
  
    *a = *b;  
  
    *b = temp;  
  
}
```

- A. Swaps values using call by value
- B. Swaps values using call by reference

- C. Does nothing
- D. Results in error

☐

☐ CUET PG PYQ Style

11. ☐ CUET PYQ-style: In C, when we pass variables to a function without pointers, the values are:

- A. Passed by reference
- B. Passed by value
- C. Passed by name
- D. Not passed

☐

12. What is the main advantage of call by reference?

- A. Speed
- B. Memory
- C. Original values can be modified
- D. Cannot modify original values

☐

13. In the following code, what is passed to function?

```
void add(int a, int b);
```

```
add(3, 4);
```

- A. 3 and 4
- B. Address of 3 and 4
- C. Nothing
- D. Variables

☐

14. What does this code do?

```
void inc(int *x) {
```

```
    (*x)++;
```

```
}
```

- A. Increments value directly
- B. Increments pointer

C. Decreases value

D. Gives error



15. If you want a function to change the original value of a variable, which method should be used?

A. Call by value

B. Call by reference

C. Static variable

D. None



☐ Advanced Concepts

16. Which of these is **not** a parameter passing method?

A. Call by value

B. Call by reference

C. Call by name

D. Call by storage



17. What is the major **disadvantage of call by reference**?

A. Uses more memory

B. Slower than call by value

C. Risk of accidentally changing original values

D. Requires recursion



18. Java uses which parameter passing method?

A. Call by value only

B. Call by reference

C. Both

D. Call by name



19. Which data structure uses recursion and parameter passing heavily?

A. Queue

B. Tree

C. Graph

D. Array

☐

20. When passing an array to a function in C, what is actually passed?

A. Array size

B. Array elements

C. Base address

D. Copy of array

☐

☐ Programs and Outputs

21. What will be the output?

```
void test(int a) {
```

```
    a++;
```

```
}
```

```
int main() {
```

```
    int x = 5;
```

```
    test(x);
```

```
    printf("%d", x);
```

```
}
```

A. 6

B. 5

C. Error

D. 0

☐

22. To swap values using a function in C, which is required?

A. Array

B. Pointer

C. Structure

D. Loop

☐

23. Which one is true for passing structure to a function?

- A. Can only be passed by value
- B. Can be passed by value or reference (using pointer)
- C. Cannot be passed
- D. Always copied

☐

24. Function overloading affects parameter passing in:

- A. C only
- B. Java
- C. C++
- D. Python

☐

25. What is the result of:

```
void change(int *x) {
```

```
    *x = 20;
```

```
}
```

```
int main() {
```

```
    int a = 5;
```

```
    change(&a);
```

```
    printf("%d", a);
```

```
}
```

- A. 5
- B. 20
- C. Error
- D. Undefined

☐

☐ Miscellaneous

26. In call by reference, the function parameter:

- A. Is a local copy
- B. Directly refers to caller's variable
- C. Cannot change value

D. Must be global

☐

27. Can functions return pointers in C?

A. No

B. Yes, but not safe

C. Yes

D. Only in main

☐

28. How are **strings** passed to functions in C?

A. By value

B. By copy

C. As pointer to character array

D. By address and size

☐

29. What happens if a function modifies a pointer passed by value?

A. Original pointer changes

B. No change in original pointer

C. Error

D. Null

☐

30. The statement `fun(&x);` suggests:

A. Call by reference

B. Call by value

C. Error

D. Recursion

☐

☐ Answer Key

1-B 2-B 3-B 4-B 5-D

6-B 7-B 8-C 9-B 10-B

11-B 12-C 13-A 14-A 15-B

16-D 17-C 18-A 19-B 20-C

21-B 22-B 23-B 24-C 25-B

26-B 27-C 28-C 29-B 30-A
